Real Estate AVM

Complete Viva Study Guide (updated): theory to code

A fair-price checker for Bangalore flats, and a test of whether location and nearby amenities really move the price

Sherlyn — Data pipeline & React frontend
Manasa — Modeling, SHAP & FastAPI backend

OJT Project · November 2026

Python · pandas · XGBoost · SHAP · FastAPI · React · Vite · Leaflet · OpenStreetMap

Contents

How to Use This Guide 5

The 30-second pitch (memorise this) . . . 5

The 3-minute explanation (structure) . . . 5

Part 1: The Big Picture 7

What is an AVM? . . . 7

The research question . . . 7

From prediction to a fair-price checker (be ready to explain this) . . . 7

Scope change . . . 7

The team and who owns what . . . 7

The dataset . . . 7

Architecture (how the pieces connect) . . . 8

Folder structure (final) . . . 8

Tech stack in one table . . . 9

Part 2: Theory You Must Know 10

2.1 Machine learning basics . . . 10

2.2 Train/test split . . . 10

2.3 Overfitting, underfitting and the bias-variance tradeoff . . . 10

2.4 Cross-validation (CV) . . . 10

2.5 Data leakage (very important) . . . 10

2.6 Missing values and imputation . . . 11

2.7 Outliers and quantile trimming . . . 11

2.8 Feature engineering we did . . . 11

2.9 Log transform (used for the linear models) . . . 11

2.10 Feature scaling . . . 11

2.11 Linear regression, Ridge and Lasso . . . 12

2.12 Decision trees . . . 12

2.13 Ensembles: bagging and Random Forest . . . 12

2.14 Boosting, gradient boosting and XGBoost . . . 12

2.15 Hyperparameter tuning . . . 13

2.16 Evaluation metrics (know the formulas) . . . 13

2.17 The baseline . . . 13

2.18 Explainability: SHAP . . . 13

2.19 Geography: geocoding, OpenStreetMap and haversine . . . 14

2.20 Fuzzy string matching (location names) . . . 14

2.21 Web and API concepts . . . 14

2.22 Home loan EMI (app feature) . . . 15

2.23 Time adjustment: bringing 2017–19 prices to 2026 . . . 15

2.24 Building age: the valuer's depreciation rule . . . 15
2.25 Home loan rules used in the EMI planner . . . 15
2.26 Front-end concepts (React) . . . 16
2.27 Designing for everyone (accessibility and plain language) . . . 16
Part 3: Sherlyn's Part (Data Pipeline and React Frontend) 17
3.1 load_bangalore.py: clean and geocode . . . 17
3.2 compute_amenity_distances.py: what is nearby . . . 17
3.3 build_final_dataset.py: merge . . . 18
3.4 clean_location_names.py: name quality analysis . . . 18
3.5 prepare_model_data.py: shared preparation (used by Manasa's scripts) . . . 18
3.6 fetch_amenity_names.py: names of the nearest places . . . 19
3.7 build_price_adjustment.py: the 2026 price factors . . . 19
3.8 The React frontend (frontend/) . . . 20
Part 4: Manasa's Part (Models, SHAP and the FastAPI Backend) 22
4.1 train_baseline_and_linear.py: baseline and linear models . . . 22
4.2 train_random_forest_xgboost.py: the main experiment . . . 23
4.3 tune_xgboost.py: tuning and saving the served model . . . 23
4.4 measure_error.py: honest error for the app . . . 23
4.5 explain_model.py: SHAP . . . 24
4.6 backend/schemas.py: Pydantic models . . . 24
4.7 backend/main.py: the FastAPI app . . . 25
4.8 backend/services/: the logic behind the endpoints . . . 26
4.9 backend/config.py and configs/config.yaml . . . 26
4.10 Tests (tests/ folder, pytest): 73 tests . . . 26
Part 5: Results and What They Mean 28
5.1 The main experiment: does location help? . . . 28
5.2 How to read these results . . . 28
5.3 Final model accuracy (tuned XGBoost, test set) . . . 28
5.3b A worked example from the live API . . . 29
5.4 What drives price (SHAP) . . . 30
5.5 The one-paragraph conclusion (say this at the end of the viva) . . . 31
Part 6: The Languages and Libraries 32
6.1 Python: why, and the basics used in this project . . . 32
6.2 pandas . . . 32
6.3 NumPy . . . 33
6.4 scikit-learn . . . 33
6.5 XGBoost, joblib and SHAP . . . 34
6.6 Geo and HTTP libraries . . . 34

6.7 FastAPI, Pydantic and Uvicorn . . . 34
6.8 JavaScript, React, Vite and Leaflet (the website) . . . 34
6.9 Matplotlib . . . 35
6.10 YAML, .env and requirements . . . 35
6.11 Virtual environment, pip, git and pytest . . . 35

Part 7: How to Run and Demo **36**

7.1 First-time setup . . . 36
7.2 Run everything and check it works . . . 36
7.3 Start the app . . . 36
7.4 Deployment (the link for mentors) . . . 36
7.5 Demo script (about 5 minutes, practise this) . . . 36
7.6 If something goes wrong in the demo . . . 37

Part 8: Limitations and Future Work **38**

8.1 Limitations . . . 38
8.2 Future work . . . 38

Part 9: Viva Question Bank **40**

9.1 Project and motivation . . . 40
9.2 Data . . . 40
9.3 Features and leakage . . . 41
9.4 Models and theory . . . 42
9.5 Metrics and results . . . 43
9.6 SHAP and explainability . . . 44
9.7 Backend (FastAPI) . . . 44
9.8 Frontend (React) . . . 46
9.8b The new features . . . 46
9.9 Python and code . . . 47
9.10 Testing, tools and process . . . 48
9.11 Tricky and "what if" questions . . . 48

Part 10: Cheat Sheet and Glossary **50**

10.1 Numbers to remember . . . 50
10.2 Commands . . . 51
10.3 Glossary . . . 51

How to Use This Guide

This guide covers the whole Real Estate AVM project, from the theory behind it to every file of code, so that both of you can answer any viva question, including questions about the other person's part. Examiners often ask a team member about work they did not do, to check they understand the whole project.

What changed since the first version of this guide. The app is now a **fair-price checker**: a React website (not Streamlit) that talks to a FastAPI backend. Prices are brought up to **2026 levels**. New features: paste-an-ad, exact location, 5 nearest named places with a map, a building-age guide, a typo check for Lakhs/Crore, a loan EMI planner with RBI rules, and a "Before you buy" checklist. All website text is in **simple English** so anyone can use it. The model and the research results did not change.

- **Part 1** gives you the pitch to memorise and the big picture.
- **Part 2** is the theory: machine learning, statistics, geography, APIs, React and the new rules (time adjustment, building age, loans).
- **Part 3** is Sherlyn's part (data pipeline and React frontend), file by file.
- **Part 4** is Manasa's part (models, SHAP and the FastAPI backend with its services), file by file.
- **Part 5** is the results and what they mean.
- **Part 6** is the languages and libraries: Python, JavaScript, React, pandas, XGBoost, FastAPI and more.
- **Part 7** is how to run, demo and deploy the project.
- **Part 8** is limitations and future work. Examiners love asking about these.
- **Part 9** is a bank of viva questions with model answers.
- **Part 10** is a one-page cheat sheet of every number, plus a glossary.

Study plan: read Parts 1 and 5 first (the story and the results), then your own part (3 or 4), then the other person's part, then Part 9 out loud to each other. Keep Part 10 open during the last revision.

The 30-second pitch (memorise this)

"Our project is an Automated Valuation Model, or AVM, for Bangalore flats, built as a **fair-price checker**. You find a flat on a property website, paste the ad or type its details, and we tell you a fair price at today's market level, whether the seller's price is too high, fair or low, and why. Underneath is a tuned XGBoost model trained on 11,272 cleaned Bangalore listings, with location and nearby-amenity features from OpenStreetMap. Our research question was whether location really improves prediction: it does, cutting the error by 18%, and SHAP shows it is about 30% of the price effect. The model is about 24% off on average. We bring its 2017–19 prices up to 2026 with published area rates, explain every estimate with SHAP, and serve it through a FastAPI backend and a simple React website."

The 3-minute explanation (structure)

1. **Problem.** Buyers cannot tell if a seller's price is fair. Manual valuation is slow. An AVM gives an instant, explained estimate.
2. **Question.** Does location and amenity data improve prediction beyond property attributes alone, and by how much?
3. **Data.** Kaggle Bengaluru House Price Data: 13,320 listings, 9 columns. We cleaned it, geocoded the area names with Nominatim, and measured distances to the nearest school, hospital and transit stop with the OpenStreetMap Overpass API.

4. **Method.** A simple baseline (area rate x size), then linear models, then Random Forest and XGBoost, each with and without location features, on the same 80/20 split. XGBoost was tuned with RandomizedSearchCV and explained with SHAP.
5. **Result.** Without location, every model loses to the simple baseline. Adding location cuts XGBoost's RMSE by 18% (58.95 to 48.28 Lakhs) and R^2 rises from 0.60 to 0.73. Size has the biggest single effect, then the area's price level.
6. **Bringing it to today.** The data is from about 2017–19. For each area we divide today's published asking rate (Home Bazaar, October 2026) by the area's old median rate, shrink noisy areas toward the city figure, and multiply the estimate by that factor (Whitefield x2.49, city x1.84).
7. **Product.** A React website with six pages: Check price, Compare, Places nearby, Other areas, Before you buy, Loan EMI. FastAPI does all the work: the model, the explanations, the area data and the tips.
8. **Limits.** Listing (asking) prices, not registered sale prices. Area-level locations. No building age, floor or condition in the data. The 2026 adjustment is area-level.

Part 1: The Big Picture

What is an AVM?

An **Automated Valuation Model** is software that estimates a property's market value using data and statistics instead of a human valuer. Real examples: Zillow's "Zestimate" in the US, and the valuation tools used by banks and portals such as MagicBricks and 99acres. Banks use AVMs to check loan amounts, and buyers use them to check whether an asking price is fair.

The theory behind AVMs is the **hedonic pricing model** from economics: a property's price is the sum of the values of its features (size, rooms, location, nearby services). Machine learning is a flexible way to learn those values from data.

The research question

"Does adding location and amenity data meaningfully improve price prediction accuracy over property attributes alone, and by how much?"

Why it is a good question:

- It is **measurable**: we compare error (RMSE, MAE, MAPE, R^2) with and without the location features, on the same test rows.
- It is **useful**: it tells a buyer or a bank what actually drives price.
- It has a **clear answer**, backed by numbers and SHAP.

From prediction to a fair-price checker (be ready to explain this)

The first version of the app listed "similar properties" from the dataset as if they were for sale. That was misleading: the records are anonymous, from 2017–19, with no photos, owner or address. So we repositioned the product:

- **We check prices, we do not list flats.** Users find flats on property websites and check them with us.
- **Old records are shown only as comparisons**, clearly labelled "old records, not flats for sale".
- **Prices are brought up to today** with published 2026 area rates (time adjustment).
- **A "Before you buy" checklist** covers what a model cannot check: whether the seller and papers are genuine.

Scope change

The original plan also wanted to test the model against market surges, such as the 2020–2022 boom. That needs dated, multi-year transaction records, which no Indian city publishes in bulk for free. So the time question was dropped from the research, and instead we bring estimates to today with a simple, documented time adjustment (Part 2.23).

The team and who owns what

Person	Owns
Sherlyn	Data and frontend: cleaning, geocoding, amenity distances and names, 2026 price factors, location name analysis, the React website
Manasa	Models and backend: baseline and linear models, Random Forest and XGBoost, tuning, error measurement, SHAP, the FastAPI API and its services (tips, area data, building age)

The dataset

- **Name:** Bengaluru House Price Data, on Kaggle (licence: Apache 2.0, free to use).
- **Size:** 13,320 rows (listings), 9 columns.
- **Columns:** `area_type`, `availability`, `location`, `size`, `society`, `total_sqft`, `bath`, `balcony`, `price`.
- **Price unit:** Lakhs. 1 Lakh = 1,00,000 rupees. 1 Crore = 100 Lakhs.
- **Messy parts:** `size` is text such as "2 BHK" or "4 Bedroom". `total_sqft` is sometimes a range ("1200-1400") or in other units ("34.46Sq. Meter"). `society` is missing for 5,502 rows, `balcony` for 609, `bath` for 73. Location names have spelling variations.
- **Availability:** about 79% are "Ready To Move". Others show a possession date such as "18-May", which means May 2018, not the 18th of May.
- **What it does not have:** sale dates, building age, floor, photos, exact addresses. This shapes several design decisions.

Architecture (how the pieces connect)

```
OFFLINE (run once, results saved as CSV files)
RAW CSV (Kaggle, 13,320 rows)
   |  load_bangalore.py            clean size/sqft, geocode area names (Nominatim)
   |  compute_amenity_distances.py nearest school/hospital/transit (Overpass)
   |  build_final_dataset.py       merge  -> bangalore_final_dataset.csv (11,672 rows)
   |  fetch_amenity_names.py       5 nearest NAMED places per area + 8,448 city places
   |  build_price_adjustment.py    2026 rate / old rate per area -> price_adjustment.csv
   v
MODEL SCRIPTS: baseline/linear -> RF/XGBoost -> tune -> measure error -> SHAP
   |  saves trained_model.pkl + location_lookup.csv
   v
ONLINE (while the website is open)
React website (frontend/, port 5173)
   |  fetch() calls, JSON over HTTP (CORS allowed)
   v
FastAPI backend (backend/main.py, port 8000)
   |-- the model: /predict (price, range, verdict, SHAP factors, today's price)
   |-- backend/services/: area_data, suggestions, listing_parser,
   |                      exact_location, building_age
   |-- OpenStreetMap Nominatim (only for "find exact spot")
```

Folder structure (final)

```
Real-Estate-AVM/
  configs/config.yaml        shared settings: seed, split, features, app settings
  data/raw/bangalore/        the Kaggle CSV (not committed)
  data/external/             current_rates_2026.csv (published area rates)
  data/processed/            cleaned data, places, price factors (committed)
  data_processing/           Sherlyn: cleaning, geocoding, amenities, names, factors
    load_bangalore.py, compute_amenity_distances.py, build_final_dataset.py
    fetch_amenity_names.py, build_price_adjustment.py
    clean_location_names.py, prepare_model_data.py
  backend/
    main.py                  FastAPI app: 12 endpoints
    schemas.py               request/response shapes (Pydantic)
    config.py                loads config.yaml
    services/                plain-Python logic used by the endpoints
      area_data.py           area summary, comparison, nearby areas, places
      suggestions.py         the tips
      listing_parser.py      reads details out of a pasted ad
      exact_location.py      coordinates, geocoding, distances
      building_age.py        age guide and age tips
    models/                  Manasa: training, tuning, SHAP + trained_model.pkl
  frontend/                  React website (Vite)
    index.html, package.json, vite.config.js, vercel.json
    src/main.jsx, App.jsx, api.js, format.js, styles.css
    src/components/          15 components (one per page or part of a page)
  streamlit_app/app.py       the older Streamlit version, kept as a backup
  reports/                   results CSVs, charts and this guide
  tests/                     73 pytest tests
  start_app.sh               starts backend + frontend together
  render.yaml                backend deployment settings (Render)
```

Tech stack in one table

Layer	Tool	Why we chose it
Languages	Python 3.14, JavaScript	Python for data and ML; JavaScript runs in every browser
Data	pandas, NumPy	Tables and fast maths
Geocoding	geopy + Nominatim	Free, OpenStreetMap-based, no API key
Amenities	OpenStreetMap Overpass API	Free data on schools, hospitals and bus stops
Models	scikit-learn, XGBoost	sklearn for baselines and tuning; XGBoost is top for tabular data
Explainability	SHAP (and XGBoost's built-in SHAP values)	Exact, additive explanations for tree models
Backend	FastAPI + Uvicorn + Pydantic	Fast, automatic validation and /docs page
Frontend	React 19 + Vite	Components and state; Vite builds and serves fast
Map	Leaflet + OpenStreetMap tiles	Free interactive map, no API key
Charts	Hand-made SVG in React	Light, no extra library, full control
Testing	pytest	Simple, standard Python testing
Deployment	Render (backend), Vercel (frontend)	Free tiers, auto-deploy from GitHub
Version control	git + GitHub	Track changes, collaborate, branches

Part 2: Theory You Must Know

2.1 Machine learning basics

- **Machine learning (ML):** the computer learns patterns from examples instead of being given rules.
- **Supervised learning:** every example has the right answer (the **label** or **target**). We have the price for each listing, so this is supervised.
- **Regression vs classification:** regression predicts a number (price). Classification predicts a category (spam or not spam). Ours is **regression**.
- **Features (X):** the inputs, such as bhk, bath, total_sqft_clean, lat, lon, the distances and location_price.
- **Target (y):** what we predict, here `price` in Lakhs.
- **Training:** the model adjusts itself to fit the training rows (`model.fit(X_train, y_train)`).
- **Prediction / inference:** using the trained model on new rows (`model.predict(X_new)`).

2.2 Train/test split

We hold back 20% of rows that the model never sees during training. We measure error on those rows to see how the model does on **new** data.

- Our split: 80/20 with `random_state=42`, giving 9,017 train and 2,255 test rows. (11,272 x 0.2 = 2,254.4, and sklearn rounds the test size up.)
- `random_state` fixes the shuffle so results are **reproducible**: everyone gets the same rows every run.
- Every model uses **the same split**, so the comparison is fair.

2.3 Overfitting, underfitting and the bias-variance tradeoff

- **Overfitting:** the model memorises the training data, noise included, and does badly on new data. Signs: very low training error, high test error. Deep trees overfit easily.
- **Underfitting:** the model is too simple to capture the pattern. High error on both. Plain linear regression underfits our data.
- **Bias** is error from wrong assumptions (too simple). **Variance** is error from being too sensitive to the training data (too complex). Good models balance the two.
- Ways we control overfitting: a test set, cross-validation, limiting `max_depth`, `subsample` and `colsample_bytree` in XGBoost, a small `learning_rate`, and averaging many trees in Random Forest.

2.4 Cross-validation (CV)

k-fold CV splits the training data into k parts. It trains on k-1 parts and validates on the remaining one, rotating k times, then averages the scores. We used **3-fold CV** inside RandomizedSearchCV, so each of the 25 settings is scored 3 times (75 fits). It gives a more reliable score than one split, and **the test set is never touched during tuning**.

2.5 Data leakage (very important)

Leakage is when information the model should not have at prediction time sneaks into training, which makes test scores look better than they really are.

In our project, `location_price` (the median price per sqft of each area) is computed **from the training rows only**. If we used all the rows, each test property's own price would be part of its area's median: the model would be "peeking" at the answer. Areas that only appear in the test set

get the **overall training median** (0.054 Lakhs/sqft = Rs 5,400/sqft). There is a pytest test that checks this.

> If asked "What is target encoding?": replacing a category (an area name) with a statistic of the target for that category (median price per sqft). It is powerful but leaks unless it is computed on training data only. That is exactly what we did.

2.6 Missing values and imputation

- `bath` missing becomes 0. (Only 54 rows, and these are often plots or land.)
- Amenity distance missing becomes the **median** distance. The median is used rather than the mean because it is not pulled by extreme values.
- Why distances are missing: no amenity was found within the 3 km search radius, or the free Overpass server timed out. 167 of 901 areas are missing all three distances, which points to failed requests rather than "no bus stop within 3 km". This is a known limitation (Part 8).

2.7 Outliers and quantile trimming

Outliers are extreme values, here often data entry mistakes (for example land in acres read as sqft: the max sqft was 1,306,800). We drop rows below the 1st or above the 99th **percentile** (quantile 0.01 and 0.99) of:

1. price per sqft, and
2. total sqft.

Both limits are computed on the untrimmed data. Result: 11,672 rows become 11,272 (400 removed). After trimming, sizes range from 500 to 5,400 sqft.

A **quantile** or **percentile** is the value below which a given % of the data falls. The median is the 50th percentile.

2.8 Feature engineering we did

- `bhk` from the text "2 BHK" (split on the space, take the number).
- `total_sqft_clean`: ranges such as "1200-1400" become the average (1300). Other units such as "Sq. Meter" or "Acres" become missing and are dropped.
- `lat`, `lon` from geocoding the area name.
- `dist_to_school_km`, `dist_to_hospital_km`, `dist_to_transit_km` from OpenStreetMap plus the haversine formula.
- `price_per_sqft = price / total_sqft_clean`, used for trimming and for location_price.
- `location_price`: the area's median price per sqft from the training rows (target encoding). This turned out to be the second most important feature.
- In the linear script only: `log1p(price)` as the target, `location_encoded` and `location_price_estimate` (= location_encoded x sqft) as an interaction feature.

2.9 Log transform (used for the linear models)

Prices are **right-skewed**: most are 50-120 Lakhs, but a few are 1,000+. Linear models fit better on `log(1 + price)`, which squashes big values. `np.log1p` does log(1+x) and `np.expm1` reverses it (e^x - 1). Predictions were clipped to the training range before reversing, to avoid huge values.

2.10 Feature scaling

`StandardScaler` turns each feature into (x - mean) / standard deviation, so the mean is 0 and the standard deviation is 1. It is needed for linear models with regularisation (Ridge and Lasso),

because the penalty treats all coefficients equally, so features must be on the same scale. **Tree models do not need scaling**, because they only compare values against thresholds.

2.11 Linear regression, Ridge and Lasso

- **Linear regression (OLS):** price = b0 + b1·x1 + b2·x2 + ... It finds the coefficients that minimise the sum of squared errors.
- **Ridge (L2 regularisation):** adds alpha × (sum of squared coefficients) to the loss. It shrinks coefficients and handles correlated features (bhk and bath are correlated). We used alpha = 1.0.
- **Lasso (L1 regularisation):** adds alpha × (sum of absolute coefficients). It can push some coefficients to exactly 0, which acts as **feature selection**. We used alpha = 0.01.
- **Why they did badly here:** price per sqft changes hugely and non-linearly between areas, and lat/lon have no straight-line relationship with price. A straight-line model cannot capture this. All the linear models were *worse* than the simple baseline.

2.12 Decision trees

A **decision tree** asks yes/no questions about features ("is sqft > 1,450?", "is location_price > 0.07?") and splits the data at each step to make the groups as similar in price as possible (for regression it minimises the squared error in each group). A leaf predicts the average price of its group.

- Pros: handles non-linear effects and interactions, needs no scaling, robust to outliers in the features.
- Cons: a single deep tree overfits.
- **Key limit: trees cannot extrapolate.** A tree can never predict outside the range of prices it saw. For a 20,000 sqft property the prediction stays flat at the level of the largest training homes. That is why our API warns when the size is outside 500-5,400 sqft.

2.13 Ensembles: bagging and Random Forest

- An **ensemble** combines many models to get a better one.
- **Bagging** (bootstrap aggregating): train many trees, each on a random sample of rows drawn with replacement, then average their predictions. Averaging reduces variance (overfitting).
- **Random Forest** = bagging + a random subset of features at each split, so the trees are less alike. We used `n_estimators=100` trees and `random_state=42`. The trees are built **independently** (in parallel).

2.14 Boosting, gradient boosting and XGBoost

- **Boosting** builds trees **one after another**. Each new tree tries to fix the errors (residuals) of all the trees before it.
- **Gradient boosting:** each new tree is fitted to the **gradient** of the loss function. For squared error, the gradient is the residual (actual - predicted). The final prediction is the starting value plus the sum of all the trees × the learning rate.
- **XGBoost** ("eXtreme Gradient Boosting") is a fast, regularised implementation. It adds L1/L2 penalties on the leaf weights, handles missing values natively, uses second-order gradients, and runs on many CPU cores.
- **Hyperparameters we tuned:**
  - `n_estimators`: how many trees (100-800 tried)
  - `max_depth`: how deep each tree can go (3-10). Deeper means more complex and more risk of overfitting
  - `learning_rate` (eta): how much each tree counts (0.01-0.2). Smaller is slower but usually generalises better

  - `subsample`: the share of rows each tree sees (0.7-1.0). Adds randomness that fights overfitting
  - `colsample_bytree`: the share of features each tree sees (0.7-1.0)
- **Our tuned model:** n_estimators = 100, max_depth = 6, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 1.0, objective = reg:squarederror, random_state = 42.
- **Bagging vs boosting in one line:** bagging means many independent trees averaged (it reduces variance). Boosting means trees built in sequence that correct each other (it reduces bias).

2.15 Hyperparameter tuning

- **Parameters** are learned from the data (tree splits, coefficients). **Hyperparameters** are set before training (max_depth, learning_rate).
- **Grid search** tries every combination. Our grid has 5 × 6 × 4 × 4 × 4 = 1,920 combinations, which is too slow.
- **Randomized search** tries a random sample, in our case `n_iter=25`, with 3-fold CV, so 75 fits. It is usually almost as good as a grid search for a fraction of the time.
- `scoring="neg_root_mean_squared_error"`: sklearn always **maximises** the score, so error is given as a negative number.
- `n_jobs=-1` uses all CPU cores. `search.best_estimator_` is the best model, refitted on the full training set.
- We saved the tuned model **only if it beat the default model on the test set**.

2.16 Evaluation metrics (know the formulas)

Let y be the actual price, ŷ the predicted price, and n the number of test rows.

- **MAE (Mean Absolute Error)** = average of |y - ŷ|. In Lakhs. "On average we are off by X Lakhs." Easy to explain.
- **RMSE (Root Mean Squared Error)** = square root of the average of (y - ŷ)². In Lakhs. Squaring punishes **big errors** more, so RMSE ≥ MAE always. This was our main metric for comparing models.
- **MAPE (Mean Absolute Percentage Error)** = average of |y - ŷ| / y × 100. A percentage, so it is fair across cheap and expensive homes. Ours is 23.7%. It is used for the price range in the app.
- **R² (coefficient of determination)** = 1 - (sum of squared errors / total sum of squares around the mean). The share of price variation explained. 1 is perfect, 0 is no better than always predicting the mean, and it can be negative.
- **Within-X%:** the share of estimates within 10% or 20% of the real price. This is the industry-standard way to judge AVMs.
- **Improvement vs baseline** = (1 - RMSE_model / RMSE_baseline) × 100.

2.17 The baseline

A **baseline** is the simplest sensible prediction, to check that the ML model actually adds value. Ours is: **area median price per sqft × property size**. This is roughly what a broker does mentally. It is strong ($R^2 \approx 0.65$), which is why beating it clearly is meaningful.

2.18 Explainability: SHAP

- **Why:** buyers and banks want to know *why* a price is what it is, not just the number. Regulators also prefer explainable models.
- **Shapley values** come from cooperative game theory (Lloyd Shapley, Nobel Prize 2012). They fairly share a "payout" (the prediction) among "players" (the features) by averaging each feature's contribution over all possible orders of adding the features.

- **SHAP** (SHapley Additive exPlanations, by Lundberg and Lee, 2017) applies this to ML models.
- **Additivity (key property):** base value + the sum of all the SHAP values = the model's prediction for that row. The **base value** (expected_value) is the average prediction over the training data.
- **TreeExplainer** computes exact SHAP values for tree models quickly, in polynomial time instead of exponential.
- **Global vs local:** *global* means the average |SHAP| per feature across all rows (our importance table and bar chart). *Local* means the SHAP values for one property (our "Why this price?" card).
- **Our global result:** size 39.55 Lakhs average effect, area price level 18.28, bathrooms 11.01, bedrooms 3.71, latitude 1.64, longitude 1.48, hospital distance 1.06, school 0.79, transit 0.54. Property features make up 69.5% of the effect and location 30.5%.
- **Important caveat:** SHAP shows what the *model* relies on, not real-world cause and effect. Correlated features (bhk, bath and sqft) can share credit.

2.19 Geography: geocoding, OpenStreetMap and haversine

- **Geocoding:** turning a place name into coordinates (latitude, longitude). We used **Nominatim**, OpenStreetMap's free geocoder, through **geopy**. Its usage policy allows 1 request per second, hence `time.sleep(1)`, plus a custom `user_agent`.
- We geocode each **unique area once** (about 1,250 areas) instead of each row (12,900+), which saves hours. 348 areas could not be found, and their rows were dropped.
- **OpenStreetMap (OSM):** a free, crowd-sourced map of the world. Places are tagged, for example `amenity=school`, `amenity=hospital`, `amenity=clinic`, `highway=bus_stop`, `railway=station`.
- **Overpass API:** a read-only query service for OSM data. Our query asks for all schools, hospitals, clinics, bus stops and railway stations `around:3000` metres (3 km) of each area. We wait 3 seconds between requests to be polite to the free server.
- **Haversine formula:** the great-circle distance between two points on a sphere:
  - $a = \sin^2(\Delta lat/2) + \cos(lat1)\cdot\cos(lat2)\cdot\sin^2(\Delta lon/2)$
  - $distance = 2\cdot R\cdot \text{asin}(\sqrt{a})$, with R = 6,371 km (Earth's radius)
  - The angles must be in **radians** (`math.radians`).
- Euclidean (straight-line) distance on lat/lon is wrong, because one degree of longitude shrinks as you move away from the equator.

2.20 Fuzzy string matching (location names)

`difflib.get_close_matches` uses `SequenceMatcher`, which scores how similar two strings are (0 to 1) from their longest matching blocks. We flag a rare name (seen 5 times or fewer) as a likely typo of a more common name when the similarity is at least **0.85**. Example: "Nagondanahalli" (5 times) versus "Margondanahalli" (26 times). This found 163 suspect pairs. Basic cleanup (strip spaces and commas, title case) merged 33 duplicate names in the raw data. **Note:** this was an analysis step. The mapping has not yet been applied to the training data (Part 8).

2.21 Web and API concepts

- **Client-server:** the React website (client, in the browser) sends requests, and the FastAPI backend (server) answers.
- **HTTP methods:** `GET` reads data (/health, /areas, /areas/{area}/summary). `POST` sends data to be processed (/predict, /advice, /parse-listing, /locate send details in the body).
- **JSON:** the text format for the data, for example `{"location": "Whitefield", "size_sqft": 1200}`.
- **Status codes:** 200 OK, 404 Not Found (unknown location), 422 Unprocessable Entity (validation failed, for example negative sqft), 500 Server Error.

- **REST API:** a style of web API built on URLs ("resources") and HTTP methods.
- **Why separate the backend and frontend?** The model loads once, and any client (a web app, a mobile app or another service) can use the same API. Each part can change independently. This is called separation of concerns.
- **Uvicorn** is an **ASGI** server (Asynchronous Server Gateway Interface) that runs the FastAPI app.

2.22 Home loan EMI (app feature)

EMI = P × r × (1 + r)^n / ((1 + r)^n - 1), where P is the loan amount, r the monthly interest rate (annual % / 12 / 100) and n the number of months. Example: an 80 Lakh price with a 20% down payment gives P = 64 Lakhs. At 8.5% over 20 years, r = 0.0070833 and n = 240, so the EMI is about Rs 55,540 a month.

2.23 Time adjustment: bringing 2017–19 prices to 2026

Our listings are from about 2017–19, and Bangalore prices have roughly doubled since. The model learned the **pattern** (how size, rooms and area shape price) from old data, so we keep the model and scale its answer:

- **Raw factor** for an area = today's published average asking rate ÷ the area's median rate in our data. Source: Home Bazaar "Property Rates in Bangalore", updated 22 October 2026: 66 localities, of which 64 match areas in our data.
- **Shrinkage:** one area's figure can be noisy, so it is pulled toward the city-wide factor: `weight = n / (n + 20)`, `factor = weight × raw + (1 − weight) × city`, where n is the number of listings in that area. An area with 20 listings is trusted half-way.
- **Clipping:** factors are kept between ×1.0 and ×3.0. Areas without a published rate use the city factor, **×1.845** (median of the area ratios).
- **Today's price** = model price × factor. Examples: Whitefield ×2.49, Electronic City ×1.60, Jayanagar ×1.69.
- This is a standard idea in valuation called a **time adjustment** (like indexing old sales with a price index).
- **Honest caveat:** the published rates are **asking** prices, which usually run above final registered sale prices (often by about a quarter). So our "today" prices are at asking-price level.

2.24 Building age: the valuer's depreciation rule

Our data has no building age, so the model cannot learn its effect. Instead we apply the rule Indian registered valuers use, and show it **separately** from the model's price:

- **Straight-line depreciation:** depreciation % = age ÷ life × (100% − salvage). For an RCC building: life 60 years, salvage 10%. A 20-year-old building: 20 ÷ 60 × 90% = **30%**.
- **Only the building wears out, not the land.** So we apply it to the cost of rebuilding the flat, not to the whole price: rebuild cost = size × ₹2,500 per sqft (standard construction in Bangalore costs about ₹2,100–2,600 per sqft in 2026).
- Example: 1,200 sqft, 20 years old: rebuild ₹30 L × 30% = ₹9 L less. ₹1.57 Cr becomes about ₹1.48 Cr.
- The age also changes the tips: still being built (check RERA, pay in stages), new (ask for the OC), 6–15 years (upkeep), 16–30 (leaks, lift, pipes), 30+ (engineer's check, smaller bank loan).

2.25 Home loan rules used in the EMI planner

- **EMI formula** (2.22 above).
- **RBI loan-to-value limits:** a bank can lend at most **90%** of the price for homes up to ₹30 L, **80%** for ₹30–75 L, and **75%** above ₹75 L. So the minimum down payment is 10%, 20% or 25%.

Stamp duty and registration are extra and are not covered by the loan.

- **Affordability rule of thumb:** advisers suggest keeping the EMI within about **30%** of take-home salary; above **40%** is too stretched (banks may approve up to 50–55%, but that leaves little for savings).
- **Best plan logic:** among 10, 15, 20, 25 and 30-year loans, pick the **shortest whose EMI is within 30%** of salary, because a shorter loan pays far less total interest. If none fits, pick the shortest within 40% and call it "tight"; if none fits that either, say it is too costly and show the salary needed.

2.26 Front-end concepts (React)

- **Single-page app (SPA):** the browser loads one page once; JavaScript then swaps the content when you change pages, without reloading. We keep the page in the address (`#check`, `#compare`) so the back button and links work (**hash routing**).
- **Component:** a JavaScript function that returns what to show (JSX). Example: `<PriceScale estimate={157} asking={175} />`. Inputs are called **props**.
- **State (`useState`):** data a component remembers, such as what the user typed. When state changes, React redraws that part of the page.
- **Effects (`useEffect`):** code that runs after drawing, for example fetching the area summary from the API when the chosen area changes.
- **Lifting state up:** the flat details live in `App.jsx` and are passed to every page, so all pages talk about the same flat.
- `fetch` **and** `async/await`**:** how the browser calls the API and waits for the answer without freezing the page.
- **CORS (Cross-Origin Resource Sharing):** the website (port 5173 or its own domain) and the API (port 8000 or Render) are different "origins". Browsers block such calls unless the API says they are allowed, which FastAPI's `CORSMiddleware` does.
- **Build step:** Vite turns our JSX files into plain, small JavaScript files in `dist/` that any web host can serve.

2.27 Designing for everyone (accessibility and plain language)

- **Plain language:** short sentences, everyday words ("Seller's price", "Price is too high"), no jargon on screen. Explanations are one tap away ("Why this price?") instead of paragraphs.
- **Progressive disclosure:** show the answer first, details on request (top 3 tips, "Show more tips", "Compare loan years").
- **Guarding against mistakes:** a Lakhs/Crore switch, and a typo check: if the seller's price is under 30% or over 3 times our estimate, we ask "Did you mean ₹3 Cr?" instead of calling it underpriced.
- **Accessibility:** every input has a label, keyboard focus is visible, colour is never the only signal (each verdict also has words), the area list works with arrow keys, and motion is reduced for users who ask for it.

Part 3: Sherlyn's Part (Data Pipeline and React Frontend)

The pipeline runs in this order. Every script is run from the project folder, because the file paths are relative to it.

```
python data_processing/load_bangalore.py            (about 25 minutes: geocoding)
python data_processing/compute_amenity_distances.py (about 1 hour: Overpass)
python data_processing/build_final_dataset.py
python data_processing/clean_location_names.py      (analysis report)
python data_processing/fetch_amenity_names.py --bulk (names of nearby places)
python -m data_processing.build_price_adjustment     (2026 price factors)
```

The processed CSVs are committed to git, so nobody has to repeat the slow geocoding and Overpass steps.

3.1 load_bangalore.py: clean and geocode

What it does, step by step:

1. `pd.read_csv` loads the raw file (13,320 rows).
2. `get_bhk(size_text)` takes "2 BHK" or "4 Bedroom", splits on the space and takes the first part as a float. It returns `None` if the value is missing or not a number.
3. `get_sqft(sqft_text)`: if the text has a "-" it is a range, so it returns the average of the two numbers. Otherwise it converts to a float. Text with units ("34.46Sq. Meter", "4125Perch", "5.31Acres") fails the conversion, so it returns `None`. 201 rows were ranges and 46 had other units.
4. `dropna(subset=[...])` drops rows missing bhk, sqft, price or location, leaving 13,257.
5. **Sanity rule:** keep only rows with at least 200 sqft per bedroom (`total_sqft_clean / bhk >= 200`). A "5 BHK in 600 sqft" listing is a data error. This leaves 12,917.
6. `str.strip()` removes extra spaces from location names.
7. **Geocoding:** for each *unique* location, call `geolocator.geocode(loc + ", Bangalore, Karnataka, India")`. Adding the city and state avoids matching a place with the same name elsewhere. It waits 1 second between calls (Nominatim policy), with `timeout=10`. A `try/except` means one failed request does not crash the whole run.
8. Coordinates are saved to `bangalore_location_coords.csv`: 1,249 areas, of which 348 could not be found.
9. `lat` and `lon` are mapped onto every row, and rows without coordinates are dropped (1,245 rows), leaving **11,672 rows in 901 areas**.
10. Saves `bangalore_sales_clean.csv`.

Code points to know:

- `data["size"].apply(get_bhk)` runs a function on each value of a column.
- `location_coords` is a **dictionary**: `{area_name: (lat, lon)}`, where the value is a tuple.
- `enumerate(unique_locations)` gives (index, value) pairs, used for the progress print.
- A `lambda x: location_coords[x][0]` is a small one-line function that takes the lat from the tuple.

3.2 compute_amenity_distances.py: what is nearby

What it does:

1. Reads the area coordinates and drops the ones that were not found.
2. For each area, it builds an **Overpass QL** query that asks for nodes tagged school, hospital, clinic, bus stop or railway station within 3,000 m:

```
[out:json][timeout:40];
( node["amenity"="school"](around:3000,LAT,LON);
  node["amenity"="hospital"](around:3000,LAT,LON);
  node["amenity"="clinic"](around:3000,LAT,LON);
  node["highway"="bus_stop"](around:3000,LAT,LON);
  node["railway"="station"](around:3000,LAT,LON); );
out body;
```

3. Sends it with `requests.post(url, data={"data": query}, timeout=50)` to a public Overpass server (overpass.openstreetmap.fr).
4. Loops over every returned element and computes the **haversine** distance from the area centre. It keeps the **minimum** distance for each category:
   - school: `amenity == school`
   - hospital: `amenity` in hospital or clinic
   - transit: `highway == bus_stop` or `railway == station`
5. If nothing is found (or the request failed), the distance stays `None`.
6. It sleeps 3 seconds between areas, then saves `bangalore_amenity_distances.csv` (901 areas).

Code points: `%f` string formatting puts the lat/lon into the query. `el.get("tags", {})` uses `.get` with a default so a missing key does not crash. The "keep the smallest" pattern is `if school_dist is None or d < school_dist: school_dist = d`.

Medians in the final data: school 0.67 km, hospital 0.27 km, transit 0.17 km.

> Possible question: "Why the area centre and not each house?" The dataset only has area names, not addresses, so area level is the most precise location we have. Every listing in an area shares the same distances.

3.3 build_final_dataset.py: merge

`sales.merge(amenities, on="location", how="left")` is a **left join**: it keeps every sale row and adds the three distance columns where the location matches. It prints how many distances are missing and saves `bangalore_final_dataset.csv` (11,672 rows, 16 columns). This is the file every model trains on.

Join types: inner (only matches), left (all left rows), right, outer (everything).

3.4 clean_location_names.py: name quality analysis

1. `basic_clean`: strip spaces, strip trailing commas and dots, collapse double spaces (`" ".join(name.split())`), and title case. This merged 33 duplicate names in the raw data (for example "rr nagar" and "RR Nagar").
2. **Typo search:** for each rare name (5 or fewer listings), `difflib.get_close_matches(name, common_names, n=1, cutoff=0.85)` looks for a near-identical, more common name. It found 163 suspicious pairs and saved them to `reports/location_name_suggestions.csv` for manual review.
3. Flags entries with more than 5 words as "descriptions, not places".
4. Saves the old-to-new name mapping to `reports/location_name_mapping.csv`.

Honest status: the mapping is a report and has not been applied to the training data yet. The final data still has pairs such as "Banaswadi" and "Banaswadi,". Applying it is listed as future work. If asked, say: "we flagged rather than auto-merged, because a fuzzy match can be wrong: 7th Block Jayanagar and 4th T Block Jayanagar are different places, so the suggestions need a human check."

3.5 prepare_model_data.py: shared preparation (used by Manasa's scripts)

This file was added so that every model script uses **identical** rows and steps. Before it existed, the same code was copied into four scripts, which risks small differences that make comparisons unfair.

- **Constants:** read from `configs/config.yaml`: `RANDOM_SEED = 42`, `TEST_SIZE = 0.2`, `TARGET = "price"`, plus the feature lists.
- `PROPERTY_FEATURES = ["bhk", "bath", "total_sqft_clean"]`
- `LOCATION_FEATURES = ["lat", "lon", "dist_to_school_km", "dist_to_hospital_km", "dist_to_transit_km", "location_price"]`
- `ALL_FEATURES` = property + location (the exact column order the saved model expects).
- `load_clean_data()`: fill bath with 0, fill the distances with their median, add price_per_sqft.
- `trim_outliers()`: drop the 1% tails of price per sqft and of sqft, giving 11,272 rows.
- `split_data()`: `train_test_split(test_size=0.2, random_state=42)`.
- `add_location_price_feature()`: area medians from **train only**, with the overall median as the fallback (no leakage).
- `build_location_lookup()`: one row per area with lat, lon, distances and location_price (881 areas), used by the API.
- `prepare_train_test()`: runs all of the above.

This was checked to reproduce the existing `location_lookup.csv` exactly.

3.6 fetch_amenity_names.py: names of the nearest places

The model only needs distances, but people want names ("Neev School, 317 m"). This script is **for display only**; the model is unchanged.

- **Bulk mode (`--bulk`, recommended):** one Overpass request downloads every school, hospital, clinic, bus stop, railway and Metro station inside a box around Bangalore (`CITY_BOX`: lat 12.60–13.40, lon 77.25–77.95). It is kept in `data/raw/` so it never has to be downloaded again.
- For each of the 881 areas, `pick_nearest` keeps the **5 nearest named places per group** within 3 km (named places first, duplicates removed).
- **Outputs:** `amenity_places.csv` (12,864 rows: location, group, rank, name, km, type, lat, lon) and `bangalore_places.csv` (8,448 named places in the city, used to measure from an exact spot).
- `nwr[...]` in the query means node, way or relation: hospitals are often drawn as buildings (ways), not single points.
- **Why bulk?** 881 separate requests overloaded the free public servers. One request plus local maths is faster and polite.

3.7 build_price_adjustment.py: the 2026 price factors

Implements the time adjustment from Part 2.23.

1. Reads `data/external/current_rates_2026.csv`: 66 localities with today's average asking rate (Home Bazaar, 22 Oct 2026).
2. Our rate per area = median of price × 1,00,000 ÷ sqft, using areas with at least 5 listings.
3. Matches names, with a few aliases (Kasavanahalli = Kasavanhalli, Hennur = Hennur Road).
4. raw factor = their rate ÷ our rate. City factor = median of the raw factors (**1.845**).
5. Shrinkage `weight = n / (n + 20)`, then clipping to ×1.0–×3.0.
6. Saves `data/processed/price_adjustment.csv` (64 areas plus a `_city_` row).

> If asked "why shrink?": an area with 6 listings has a shaky median, so one odd listing could double its factor. Shrinkage trusts big areas and pulls small ones toward the city, a standard way to handle small samples.

3.8 The React frontend (frontend/)

Big idea: the website only shows things and collects input. Every number comes from the API, so the website never needs the data files or the model. This is **separation of concerns**: the same API could serve a mobile app.

How it runs: `npm install` downloads React, Leaflet and Vite once. `npm run dev` starts Vite on port 5173 with instant reload. `npm run build` makes small static files in `dist/` for hosting. `VITE_API_URL` (in `.env` or the hosting settings) says where the API is.

Files:

File	What it does
`index.html`	The one HTML page; loads the fonts and `src/main.jsx`
`main.jsx`	Starts React and draws `<App />` into the page
`App.jsx`	Sidebar, page switching (hash routing), and the shared state: the flat details, the exact spot and the last check
`api.js`	Every call to the backend in one place, with a 75-second timeout (a free server can take a minute to wake) and friendly error messages
`format.js`	Indian number formats: `money(156.6)` gives "₹1.57 Cr", `rupees(123456)` gives "1,23,456"
`styles.css`	The look: sage paper, forest green, gold; sidebar layout; phone layout below 900 px

Components (one per page or part of a page):

Component	Purpose
`CheckPrice.jsx`	The main page: form, paste-an-ad, Lakhs/Crore switch, typo check, verdict, key numbers, age line, tips, "Why this price?"
`AreaPicker.jsx`	Searchable list of the 881 areas; works with arrow keys and Enter
`PriceScale.jsx`	The Cheap / Fair / Costly bar with the seller's price pin; fair = within 10% of our price
`PriceWaterfall.jsx`	SHAP waterfall: average flat, then each factor up (green) or down (red), then × the market factor = today
`Suggestions.jsx`	Tips from the API; top 3 shown, "Show more tips" for the rest
`Compare.jsx`	Histogram of price per sqft of similar flats (hand-drawn SVG), with "Our price" and "Seller's price" lines and "costlier than X% of flats"
`PlacesPage.jsx`, `NearbyPlaces.jsx`, `PlacesMap.jsx`, `ExactLocation.jsx`	Places nearby: area picker, society/Maps-link finder, 5 nearest per group, Leaflet map with OpenStreetMap tiles
`NearbyAreas.jsx`	The 10 closest areas (with at least 5 records), % cheaper or costlier, "Check here"
`ChecklistPage.jsx`, `Checklist.jsx`	Six things to do before paying, as tick boxes
`EmiPage.jsx`, `EmiCalculator.jsx`	EMI sliders, RBI down-payment check, best loan length for your salary

Key behaviours to be able to explain:

- **Check price flow:** `api.predict(flat)` gets the price; if the seller's price is under 30% or over 3× our price it is treated as a typo; then `api.advice(...)` gets the tips (with the building age and exact spot). Both results are stored in `check` in App, so other pages (Compare, EMI, Before you buy) can use them.
- **Stale result:** if the user edits the form after checking, a note says "You changed the details. Press Check price again."

- **Paste an ad:** `/parse-listing` returns area, bedrooms, bathrooms, size, price and building age; the form fills itself and says what is missing. Links cannot be read, because property sites block other apps; the user pastes the ad's text instead.
- **Lakhs/Crore:** the price is always stored in Lakhs; the switch only changes how the typed number is read (3 in Crore = 300 Lakhs).
- **Hash routing:** the page is in the address (`#emi`), so refresh and back work without a router library.
- **Simple language:** every label and message is short ("Seller's price", "Price is too high"); details are behind taps.

The Streamlit version is kept in `streamlit_app/app.py` as a backup (`bash start_streamlit.sh`). If asked why we moved: React gave us full control of the design, real page navigation, a faster feel, and a clean split between website and API, which also makes the website easy to host separately.

Part 4: Manasa's Part (Models, SHAP and the FastAPI Backend)

Run order (from the project folder, inside the venv):

```
python -m backend.models.train_baseline_and_linear     baseline + linear models
python -m backend.models.train_random_forest_xgboost   main experiment
python -m backend.models.tune_xgboost                  tune + save model + lookup
python -m backend.models.measure_error                 error used by the app
python -m backend.models.explain_model                 SHAP charts
uvicorn backend.main:app --reload                      start the API
```

`python -m package.module` runs a file *as a module*, so imports like `from data_processing.prepare_model_data import ...` work from the project folder.

4.1 train_baseline_and_linear.py: baseline and linear models

Steps:

1. `load_data()` fills missing bath and balcony with 0 and the distances with their median.
2. `print_diagnostics()` prints the price and sqft distributions. It found outliers such as a max sqft of 1,306,800, which is land read as sqft.
3. `trim_outliers()` applies the 1%/99% trim on price per sqft and sqft, leaving 11,272 rows.
4. 80/20 split (`random_state=42`).
5. **Baseline:** area median price per sqft (train only, global median fallback) × sqft.
6. The target is transformed: `y_train_log = np.log1p(y_train)`.
7. `run_condition()` trains three models, each as `make_pipeline(StandardScaler(), Model)`:
   - LinearRegression, Ridge(alpha=1.0) and Lasso(alpha=0.01)
   - Predictions are clipped to the training log range and turned back with `np.expm1`.
8. **Four feature conditions:**
   - `property_only`: bhk, bath, balcony, sqft
   - `plus_location`: + lat, lon and the 3 distances
   - `plus_location_encoded`: + location_encoded (the area median rate)
   - `plus_location_interaction`: + location_price_estimate (rate × sqft)
9. `evaluate()` computes RMSE, MAE, MAPE and R^2, and saves `reports/baseline_and_linear_results.csv`.

Results:

Model	Features	RMSE	MAE	MAPE	R^2
Baseline	area rate × size	54.82	27.72	25.6%	0.654
Lasso	property only	82.01	36.19	32.5%	0.226
Lasso	+ location (raw)	81.00	35.53	31.8%	0.245
Lasso	+ location encoded	69.53	30.98	27.7%	0.444
Linear	+ interaction	70.05	30.70	27.7%	0.436

Lessons (say these in the viva):

- **Every linear model was worse than the simple baseline.** Price does not rise in a straight line with lat/lon or distance. The area effect is *multiplicative* (rate × size), not additive.
- Adding the encoded area rate improved linear R^2 from about 0.23 to about 0.44, which is the first sign that **location matters**.
- This is why we moved to tree models, which handle non-linearity and interactions automatically.

- Code points: `make_pipeline` chains scaling and the model, so the scaler is fitted on the training data only (no leakage). The dict merge `baseline_result | {...}` is Python 3.9+ syntax.

4.2 train_random_forest_xgboost.py: the main experiment

The design (a controlled experiment): the same rows, the same split, the same models, and only the features change.

Condition	Features
property_only	bhk, bath, total_sqft_clean
plus_amenities	+ lat, lon, school, hospital, transit distances
plus_area_price	+ location_price (all 9 features, the final model's set)

For each condition it trains `RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)` and `XGBRegressor(n_estimators=100, random_state=42)`, and scores them on the test rows: RMSE, MAE, MAPE, R^2 and the % improvement over the baseline. The results are saved to `reports/model_comparison.csv` and shown in the app's How It Works tab. The script finishes by printing the headline: how much adding location cuts XGBoost's RMSE.

The results are in Part 5.

Code points:

- `score()` returns a **dictionary** per model, and a list of dicts becomes a DataFrame with `pd.DataFrame(results)`.
- f-strings such as `f"{rmse:6.2f}"` mean 6 characters wide with 2 decimals.
- `results_df.set_index("condition")` and then `.loc["property_only", "rmse"]` looks up one value.

4.3 tune_xgboost.py: tuning and saving the served model

1. `prepare_train_test()` gives the shared rows and split.
2. Baseline RMSE for reference.
3. A **default** XGBoost (100 trees) gives the RMSE to beat.
4. **RandomizedSearchCV**:
   - `param_distributions`: n_estimators [100, 200, 300, 500, 800], max_depth [3, 4, 5, 6, 8, 10], learning_rate [0.01, 0.05, 0.1, 0.2], subsample [0.7, 0.8, 0.9, 1.0], colsample_bytree [0.7, 0.8, 0.9, 1.0]
   - `n_iter=25`, `cv=3`, `scoring="neg_root_mean_squared_error"`, `random_state=42`, `n_jobs=-1`
5. `search.best_estimator_` is scored on the test rows.
6. **The model is only saved if tuned RMSE < default RMSE**: `joblib.dump(tuned_model, "backend/models/trained_model.pkl")`. The best settings go to `reports/best_params.csv`.
7. It also saves `data/processed/location_lookup.csv` (881 areas) from the *same* rows and training medians, so the model and the lookup always match.

Saved model settings: n_estimators = 100, max_depth = 6, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 1.0.

joblib: saves Python objects to disk (a pickle that is efficient for big NumPy arrays). `joblib.load` brings the trained model back in the API, so there is no retraining at startup.

4.4 measure_error.py: honest error for the app

It loads the saved model, predicts on the 2,255 test rows, and saves `reports/model_error.csv` with: MAPE 23.7%, MAE 23.96 Lakhs, RMSE 46.41 Lakhs, R^2 0.752, 33.7% of estimates within 10% of the real price and 56.7% within 20%, and the test row count. The API uses MAPE for the price range: estimate ± 23.7%.

> Why a percentage range and not a fixed ± amount? The error grows with price. ± 24 Lakhs would be huge for a 40 Lakh flat and tiny for a 3 Crore villa. A % range scales correctly.

4.5 explain_model.py: SHAP

1. Loads the model and the same prepared rows (train + test = 11,272), with location_price from the training medians.
2. `explainer = shap.TreeExplainer(model)` and `shap_values = explainer.shap_values(x)` give one value per feature per row.
3. **Global importance** = `np.abs(shap_values).mean(axis=0)`, the average absolute effect in Lakhs. The `abs` is there because we care about size, not direction. The table is saved to `reports/shap_feature_importance.csv`.
4. Groups the features into **property vs location** and saves `reports/shap_group_share.csv`: **69.5% property, 30.5% location**.
5. Charts: a bar chart (`shap_feature_importance.png`) and a **summary / beeswarm plot** (`shap_summary.png`). In the beeswarm, each dot is one property, the x-axis is the SHAP value, and the colour is the feature's value (red high, blue low).

SHAP results (average effect in Lakhs):

Feature	Effect	Group
total_sqft_clean (size)	39.55	property
location_price (area level)	18.28	location
bath	11.01	property
bhk	3.71	property
lat	1.64	location
lon	1.48	location
dist_to_hospital_km	1.06	location
dist_to_school_km	0.79	location
dist_to_transit_km	0.54	location

Interpretation: size is the biggest single driver. The area's price level is second and makes up most of the location effect. The three amenity distances add up to only about 2.4 Lakhs on average. So **"which area" matters much more than "how close to the nearest school"**. One reason: the distances are measured at area level and most areas are dense, so the distances vary little (median transit distance is 0.17 km).

Bathrooms rank above bedrooms. One reason: bath correlates with luxury and size (big villas have 4-5 baths), and bhk and bath are correlated, so the model gives more of the shared credit to bath.

4.6 backend/schemas.py: Pydantic models

Pydantic checks data types and ranges automatically. FastAPI uses these classes to validate requests and shape responses.

```
class PredictionRequest(BaseModel):
    location: str = Field(min_length=1)
    size_sqft: float = Field(gt=0, le=100000)
    bedrooms: int = Field(ge=1, le=20)
    bathrooms: float = Field(ge=0, le=20)
    asking_price: float | None = Field(default=None, gt=0)
    @model_validator(mode="after")
    def check_size_per_bedroom(self): ...   # >= 200 sqft per bedroom

class AdviceRequest(PredictionRequest):        # inherits every field above
    exact_lat: float | None; exact_lon: float | None
    building_age: int | None = Field(default=None, ge=0, le=100)
    under_construction: bool = False
```

- `gt` greater than, `ge` greater or equal, `le` less or equal. `float | None` means optional.
- A broken rule gives **422** automatically, before our code runs. The 200 sqft-per-bedroom rule is the same rule used in data cleaning.
- `PredictionResponse` has: predicted_price (2017–19 level), price_low, price_high, error_percent, price_per_sqft, verdict, difference_percent, base_price, factors, warnings, and the **today** fields: today_price, today_low, today_high, today_price_per_sqft, market_factor, market_basis ("area" or "city"), market_source.
- `ListingText` (pasted ad) and `PlaceQuery` (society name + area) are the other request shapes.

4.7 backend/main.py: the FastAPI app

At startup (runs once, keeps requests fast): loads config, the model (`joblib.load`), the location lookup, the measured error (23.7%), the market factors from `price_adjustment.csv`, and the area data (via `backend/services/area_data.py`).

SHAP without the shap library: XGBoost can compute exact SHAP values itself: `booster.predict(DMatrix(features), pred_contribs=True)` returns one contribution per feature plus a last "bias" column, which is the base price. This gives the same numbers as `shap.TreeExplainer` (a test checks this) but uses much less memory, which matters on a free server.

CORS: `app.add_middleware(CORSMiddleware, allow_origins=...)` lets the React website (a different address) call the API. `ALLOWED_ORIGINS` can limit it to the website's address.

The 12 endpoints:

Method	Path	Returns
GET	/health	status, model version, number of areas
GET	/locations	sorted list of the 881 areas
GET	/model-info	features, base price, error, data period, city factor
POST	/predict	price (old and today), range, verdict, SHAP factors, warnings
GET	/areas	every area with lat, lon and number of records
GET	/areas/{area}/summary	typical rate today, typical price for the BHK, % ready to move
GET	/areas/{area}/places	5 nearest schools, hospitals, transit stops (from centre, or from lat/lon if given)
GET	/areas/{area}/comparison	today's price per sqft of similar flats, typical and middle half
GET	/areas/{area}/nearby	10 closest areas with today's rate and % difference
POST	/parse-listing	details read from a pasted ad, what is missing, building age
POST	/locate	coordinates of a society, landmark, Maps link or "lat, lon"; km from the area centre
POST	/advice	the tips, and the building-age guide
GET	/docs	automatic interactive Swagger UI

/predict logic:

1. Unknown area: `HTTPException(404)`.
2. `build_features()` makes a one-row DataFrame in the exact training column order.
3. Model price (2017–19 level), SHAP factors (lat and lon combined into "Position on the map"), range = price ± 23.7%.
4. **Today:** `factor, basis = market_factor(location)`; today_price = price × factor (and the range too).
5. **Verdict compares the seller's price with today's price:** more than +10% is "Looks overpriced", below −10% "Looks underpriced", otherwise "Looks fairly priced". (The website shows these as "Price is too high / fair / low".)
6. Warnings for sizes outside the training range (500–5,400 sqft) or more than 10 bedrooms.

/advice logic: runs /predict, then replaces the old prices with today's prices so tips compare like with like; prices the same flat in up to 3 cheaper nearby areas with the model; gets the nearest places (from the exact spot if given); then `build_suggestions(...)`, plus the age tips and age guide from `building_age.py`.

4.8 backend/services/: the logic behind the endpoints

Plain Python and pandas, no web code, so each file can be tested without starting a server.

- **area_data.py:** reads the dataset, lookup, places and factors once. `area_summary`, `comparison` (same area and BHK; if fewer than 5 records, the whole area; then the city), `nearby_areas` (skips spelling variants such as "Whitefield," and areas with under 5 records), `places_near_area`, `places_near_point`.
- **suggestions.py:** `build_suggestions` returns short (kind, title, text) tips in a fixed order: the seller's price (offer and maximum), value against the area, size against similar flats, bathrooms, daily needs, how many flats are still being built, few records, cheaper areas, model warnings. Kinds are good, warning or info.
- **listing_parser.py:** regular expressions read "2 BHK", "1,150 sq.ft", "2 bathrooms", "Rs 85 Lac" / "1.75 Cr", and match the area against the 881 names (longest match first). `availability_text("18-May")` gives "Possession May 2018".
- **exact_location.py:** `parse_coordinates` reads "12.97, 77.75" or a Google Maps link; `geocode` asks Nominatim, limited to a box around Bangalore; `distances_km` is vectorised haversine with NumPy; `nearest_places` gives 5 per group within 3 km.
- **building_age.py:** `depreciation_share(age)` = min(age, 60) / 60 × 0.9; `age_adjustment` applies it to size × ₹2,500; `age_suggestions` gives one tip by age band; `find_building_age` reads "8 years old", "Age of property: 12 years" or "Under construction" from an ad.

> If asked "why is age not in the model?": our data has no building age, so the model cannot learn it. We use the standard valuer rule and show it separately, labelled as a rough guide. Mixing it into the model's price would hide what the model actually learned.

4.9 backend/config.py and configs/config.yaml

`config.py` finds `PROJECT_ROOT` from its own file location and loads the YAML with `yaml.safe_load` (which will not run arbitrary code, unlike `yaml.load`). The YAML holds the seed, split, features, model file, the fair range (10%), the market source, the fallback error and the training size range.

4.10 Tests (tests/ folder, pytest): 73 tests

Run with `python -m pytest -q`. `conftest.py` puts the project folder on the import path. The API tests call the endpoint functions directly, so no server is needed.

File	What it checks
test_data_preparation.py (5)	No gaps; row counts 11,672 → 11,272 → 9,017 / 2,255; trimming; **no leakage**; repeatable split
test_api.py (19)	/health, /locations; positive price with low < price < high; **SHAP adds up to the prediction**; our SHAP base equals the shap library's; bigger home costs more; 404 and 422 cases; verdict thresholds; today's price uses the factor; verdict compares with today's price; /areas matches /locations; ad reading; advice says "Ask for a lower price" when overpriced; age guide does not change the model price
test_area_data.py (6)	Areas sorted; summary brought to today; comparison scope widens when records are few; nearby areas skip variants; places within 3 km
test_suggestions.py (9)	The right tip for overpriced, fair, cheap, no price, small flat, few bathrooms, model warnings, exact-spot distances
test_listing_parser.py (9)	Reading prices in Lakhs and Crore, sizes, BHK, label-first text, links, "18-May" dates
test_exact_location.py (9)	Coordinates and Maps links, haversine, nearest places
test_amenity_names.py (11)	Choosing the 5 nearest named places, transit types, the places table
test_building_age.py (5)	20 years = 30% (valuer textbook case); stops at 60 years; only the building depreciates; tips by age; reading age from an ad

Part 5: Results and What They Mean

5.1 The main experiment: does location help?

All models were trained on the same 9,017 rows and tested on the same 2,255 unseen rows. Lower RMSE, MAE and MAPE are better, and higher R^2 is better. "vs baseline" is the % reduction in RMSE compared with the area rate × size baseline (negative means worse than the baseline).

Model	Features	RMSE	MAE	MAPE	R^2	vs baseline
Baseline	Area rate × size	54.82	27.72	25.6%	0.654	-
Random Forest	Property only	59.17	29.73	28.4%	0.597	-7.9%
XGBoost	Property only	58.95	30.06	28.7%	0.600	-7.5%
Random Forest	+ position & amenities	48.07	23.77	22.7%	0.734	+12.3%
XGBoost	+ position & amenities	47.52	24.14	23.3%	0.740	+13.3%
Random Forest	+ area price level (all 9)	49.41	23.61	22.0%	0.719	+9.9%
XGBoost	+ area price level (all 9)	48.28	23.88	22.7%	0.732	+11.9%

5.2 How to read these results

- **Random Forest:** with property features only, RMSE is 59.17 Lakhs (R^2 0.597). Adding location brings it down to 48.07 (amenities and position) and 49.41 (plus area price level), with R^2 0.719 in the full model. That is a **19% drop in RMSE** at the best location setting.
- **XGBoost:** with property features only, RMSE is 58.95 Lakhs (R^2 0.600). Adding location brings it down to 47.52 (amenities and position) and 48.28 (plus area price level), with R^2 0.732 in the full model. That is a **19% drop in RMSE** at the best location setting.
- **Property-only models lose to the baseline** (baseline RMSE 54.82). Without knowing the area, even a strong tree model cannot match a simple area-rate rule. This is the clearest evidence that location matters.
- **With location, the tree models beat the baseline.** They combine the area effect with size, bedrooms and bathrooms in non-linear ways the baseline cannot.
- **Position and amenities vs area price level:** because every area has its own unique lat/lon and distances, the trees can already recognise each area from these columns. location_price gives the same information as one ready-made number, so the step from condition 2 to condition 3 is small. Part of the "amenity" gain is really "area identity", which matches the SHAP finding that individual distances have small effects.
- **Linear models** (Part 4.1) were all worse than the baseline, which confirms the relationship is non-linear.

5.3 Final model accuracy (tuned XGBoost, test set)

Metric	Value
MAPE (average % error)	23.7%
MAE	23.96 Lakhs

The API's price range is the estimate ± 23.7% (the measured MAPE).

Tuning helped: the default XGBoost with all 9 features has RMSE 48.28 on the same test rows. The tuned model has 46.41, about **3.9% lower**, and the best R^2 of any model (0.752). That is why the script saved it.

How good is "56.7% within 20%"? More than half of all estimates land within ±20% of the listed price, and a third within ±10%. Professional AVMs with address-level data report 70-90% within

10%. Ours is a reasonable first check, not a formal valuation, and the app says so by showing a range.

5.3b A worked example from the live API

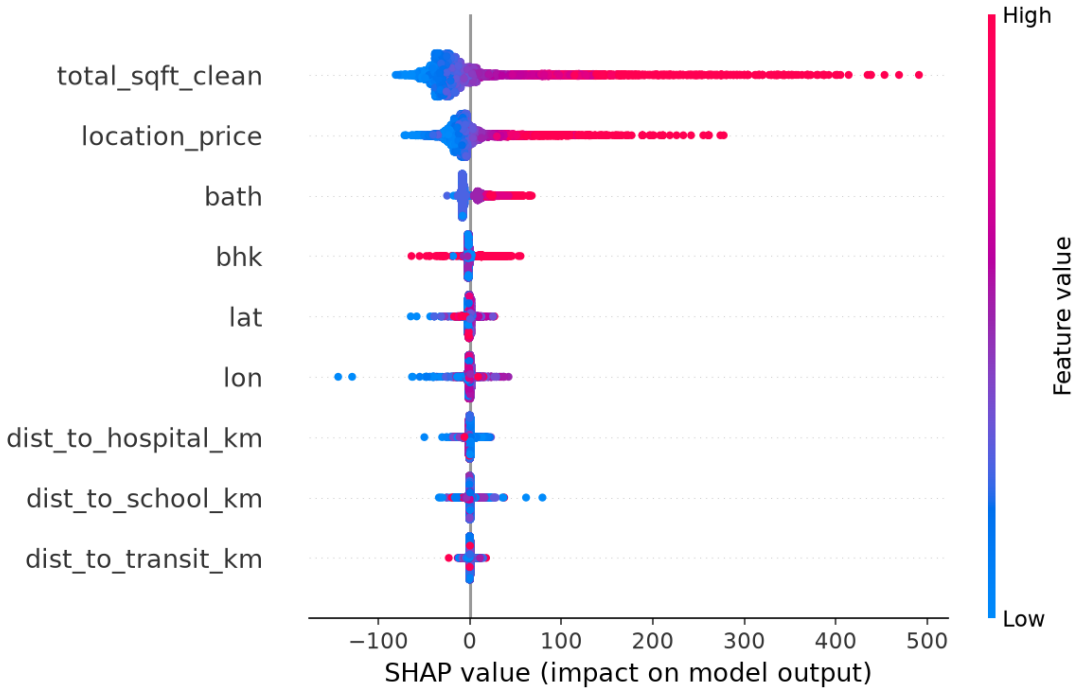
Request: Whitefield, 1,200 sqft, 2 BHK, 2 bathrooms, seller's price ₹1.75 Cr (175 Lakhs).

Output	Value
Model price (2017–19 level)	62.81 Lakhs (₹5,234 per sqft)
Market factor for Whitefield	×2.494 (its own published rate)
Our price today	**156.65 Lakhs ≈ ₹1.57 Cr** (₹13,054 per sqft)
Likely range today	119.51 – 193.78 Lakhs (± 23.7%)
Fair band (website bar)	₹1.41 Cr – ₹1.72 Cr (± 10%)
Verdict	"Looks overpriced", shown as **"Price is too high"**: the seller asks 11.7% more
Starting price (base value)	102.58 Lakhs, the average prediction
Size, 1,200 sqft	−26.35
Bathrooms, 2	−7.37
Area price level in Whitefield	−4.18
Bedrooms, 2 BHK	−1.40
Nearest hospital, 170 m	+0.60
Nearest transit, 13 m	−0.56
Nearest school, 1.1 km	−0.27
Position on the map	−0.24

102.58 − 26.35 − 7.37 − 4.18 − 1.40 + 0.60 − 0.56 − 0.27 − 0.24 = **62.81**. That is SHAP additivity in action. Then 62.81 × 2.494 = 156.65 Lakhs today.

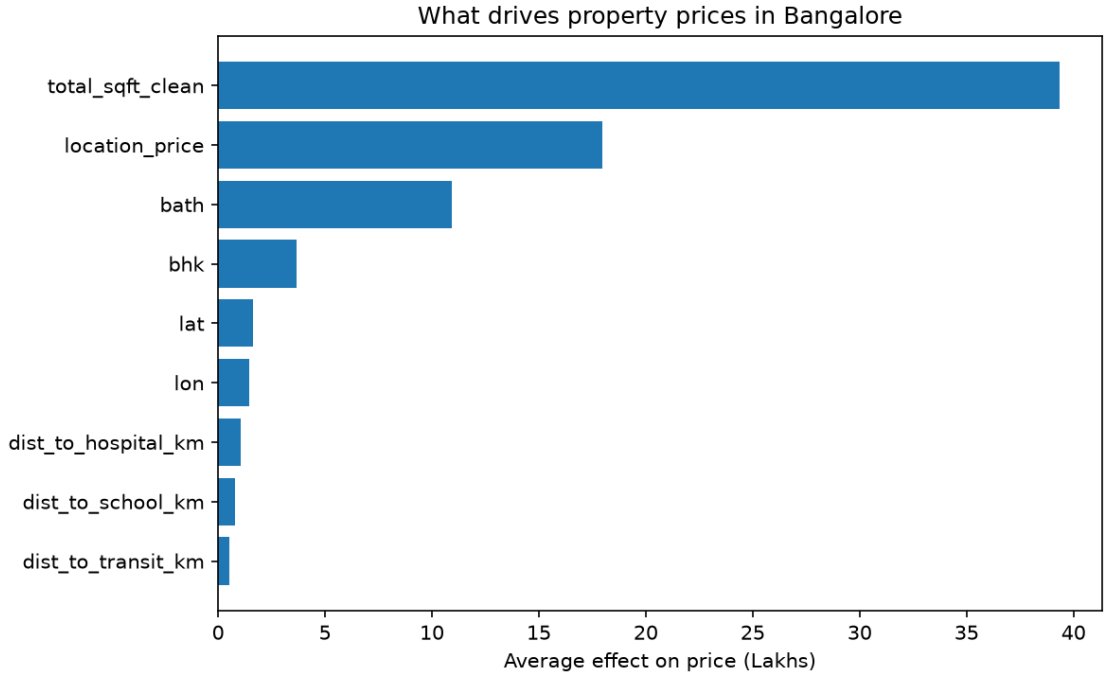
With a building age of 20 years, the website also shows "20-year-old building: about ₹1.48 Cr" (rebuild cost 1,200 × ₹2,500 = ₹30 L; 30% of that is ₹9 L).

> Why are most factors negative? SHAP compares against the *average property* (102.58 Lakhs at 2017–19 prices, a large home). A 1,200 sqft 2 BHK is smaller than average, so size and bathrooms pull the price down.



SHAP summary (beeswarm): every dot is one property; right means the feature pushed its price up, colour shows whether the feature value was high (red) or low (blue)

5.4 What drives price (SHAP)



SHAP feature importance: average effect of each feature on the predicted price, in Lakhs

- **Size** is the biggest single driver (39.55 Lakhs average effect).
- **Area price level** is second (18.28 Lakhs) and carries most of the location effect.
- **Bathrooms** (11.01) and **bedrooms** (3.71) follow.
- **Map position and the three amenity distances** are each small (under 2 Lakhs).
- **Property 69.5%, location 30.5%** of the total effect (54.27 vs 23.80 Lakhs).

5.5 The one-paragraph conclusion (say this at the end of the viva)

"Location clearly matters. A model that knows only size, bedrooms and bathrooms cannot even beat a simple 'area rate × size' rule, and adding location features cuts the error substantially. But the value of location comes from *which area* a property is in, captured by the area's price level and its position, not from the distance to the nearest school, hospital or bus stop. Those distances add little, partly because we could only measure them at area level. So for Bangalore, the old saying 'location, location, location' holds, but at the neighbourhood level."

Part 6: The Languages and Libraries

6.1 Python: why, and the basics used in this project

Why Python? It is the standard language for data science and ML, readable, and has mature libraries (pandas, scikit-learn, XGBoost, SHAP, FastAPI, Streamlit). It is **interpreted** (runs line by line, with no compile step), **dynamically typed** (variables have no fixed type), and **multi-paradigm** (procedural, object-oriented, functional). We used Python 3.14.

Data types used:

- `int` (bedrooms), `float` (sqft, price), `str` (location), `bool`, `None` (missing).
- `list`: ordered and changeable, for example `PROPERTY_FEATURES = ["bhk", "bath", "total_sqft_clean"]`.
- `tuple`: ordered and unchangeable, for example `(lat, lon)` in the geocoding dictionary.
- `dict`: key to value, for example `{"location": "Whitefield", "size_sqft": 1200}` (also the JSON shape) and the `readable_names` map in the app.

Language features used (with examples from our code):

- **Functions** with `def`, parameters, default values (`def load_clean_data(path=DATA_PATH)`) and `return`.
- **Docstrings:** the `"""triple-quoted"""` text at the top of a function or file, which documents it.
- **Lambda:** a small one-line anonymous function, for example `lambda x: str(round(x, 2)) + " Lakhs"`.
- **List comprehension:** `[n for n in all_names if name_counts[n] > name_counts[name]]` builds a list in one line.
- **f-strings:** `f"RMSE={rmse:6.2f}"` puts values into text, with formatting.
- `%` **formatting:** `"... %f ..." % (lat, lon)` is the older style, used in the Overpass query.
- **try/except:** handles errors without crashing, for example a failed geocode request. Catching a *specific* exception (`except requests.exceptions.RequestException`) is better than a bare `except:`, which also hides real bugs. We replaced the bare ones in the app.
- **if/elif/else, for loops, enumerate, zip:** `zip(features.columns, shap_values)` pairs each feature name with its SHAP value, and `dict(zip(...))` turns the pairs into a dictionary.
- **Modules and imports:** `from backend.schemas import PredictionRequest`. A folder of .py files acts as a package.
- `if __name__ == "__main__":` runs that code only when the file is run directly, not when it is imported.
- **Type hints:** `def load_data() -> pd.DataFrame:` and `float | None`. Python does not enforce them, but FastAPI and Pydantic use them for validation.
- **Classes:** `class PredictionRequest(BaseModel)` **inherits** from Pydantic's BaseModel, so it gets validation for free.
- **Decorators:** `@app.post("/predict")`, `@st.cache_data`, `@model_validator`. A decorator is a function that wraps another function to add behaviour.
- **sort with key:** `factors.sort(key=lambda f: abs(f.effect), reverse=True)` sorts by absolute effect, largest first.
- **pathlib.Path:** `Path(__file__).resolve().parent.parent` builds file paths that work on any operating system, and `/` joins path parts.
- **os.getenv:** reads an environment variable, with a default.
- **Dict merge** `|` (Python 3.9+): `baseline_result | {"condition": "baseline"}`.

6.2 pandas

pandas works with **DataFrames** (tables of rows and columns) and **Series** (a single column).

ion	What it does	Where we used it	
ad_csv / to_csv	read / write a CSV	every script	
ol"]	select a column (Series)	everywhere	
ndition]	boolean filter (mask)	trimming, filtering listings	
	`	combine conditions (with brackets)	trimming
y(func)	run a function on each value	get_bhk, get_sqft	
dict or Series)	replace values by lookup	location_price	
na(value)	fill missing values	bath, distances	
na(subset=)	drop rows with missing values	cleaning, geocoding	
pby("location")["x"].median()	a statistic per group	location_price, area rates	
tile(0.01)	percentile	outlier trimming	
e(other, on=, how=)	SQL-style join	build_final_dataset	
_duplicates(subset=)	keep the first row per key	location lookup	
index("location") + .loc[name]	fast lookup by label	API, app	
[0]	select by position	first row	
e_counts() / .mode()	counts / most common value	name analysis, area type	
strip(), .str.contains()	text operations	cleaning names	
().sum()	count missing values	checks	
ncat([a, b])	stack tables	SHAP on train + test	
ribe()	summary statistics	diagnostics	

Vectorisation: `data["price"] / data["total_sqft_clean"]` divides whole columns at once (fast C code) instead of looping over rows in Python.

6.3 NumPy

The fast numeric array library underneath pandas. We used `np.sqrt` (RMSE), `np.log1p` / `np.expm1` (log transform), `np.clip` (limit predictions), `np.abs(...).mean(axis=0)` (SHAP importance, where `axis=0` means down the columns, one result per feature), and `np.array(...).reshape(-1)` (to flatten the SHAP base value).

6.4 scikit-learn

The common API (know this): every model has `.fit(X, y)` to train and `.predict(X)` to make predictions. Transformers have `.fit`, `.transform` and `.fit_transform`.

- `train_test_split`: the 80/20 split.
- `LinearRegression`, `Ridge`, `Lasso` (from `sklearn.linear_model`).
- `RandomForestRegressor` (from `sklearn.ensemble`).
- `StandardScaler` (from `sklearn.preprocessing`).
- `make_pipeline`: chains steps so they are fitted together. It prevents leakage, because the scaler only sees the training data.
- `RandomizedSearchCV`: hyperparameter search with cross-validation.
- Metrics: `mean_squared_error`, `mean_absolute_error`, `mean_absolute_percentage_error` (returns a fraction, so multiply by 100), `r2_score`.

6.5 XGBoost, joblib and SHAP

- `XGBRegressor(...)` follows the sklearn API (`fit`, `predict`), so it works with RandomizedSearchCV.
- `joblib.dump(model, path)` / `joblib.load(path)` save and load the trained model (`.pkl`). **Caution:** only load pickle files you trust, because unpickling can run code.
- `shap.TreeExplainer(model)` gives `.expected_value` (the base) and `.shap_values(X)` (one value per feature per row). `shap.summary_plot` draws the beeswarm chart.

6.6 Geo and HTTP libraries

- **geopy:** `Nominatim(user_agent=..., timeout=10)` and `.geocode("place")` return an object with `.latitude` and `.longitude`.
- **requests:** `requests.post(url, data=... or json=..., timeout=...)`, `response.json()` and `response.status_code`. Use `json=` for JSON bodies (the app to the API), and `data=` for form data (the Overpass query).
- **math:** `radians`, `sin`, `cos`, `asin`, `sqrt` for haversine.
- **difflib:** `get_close_matches` for fuzzy name matching.
- **time:** `time.sleep(1)` for rate limiting.

6.7 FastAPI, Pydantic and Uvicorn

- **FastAPI** is a modern Python web framework. It is fast (built on Starlette, async), validates data automatically with Pydantic, and generates interactive docs at **/docs** (Swagger UI) and /redoc from the code.
- **Pydantic:** data validation through Python classes and type hints (`BaseModel`, `Field`, `model_validator`).
- **Uvicorn:** the ASGI server that runs the app: `uvicorn backend.main:app --reload`. Here `backend.main` is the module, `app` is the variable, and `--reload` restarts on code changes (for development only).
- **FastAPI vs Flask:** Flask is older and simpler but has no built-in validation or auto-docs. FastAPI gives both, plus async support.

6.8 JavaScript, React, Vite and Leaflet (the website)

JavaScript basics used in our code:

- `const` (a name that will not be re-assigned) and `let`; arrow functions `(x) => x * 2`.
- Template-free string joining with `+`, objects `{ location: 'Whitefield' }`, arrays and `.map()` to turn a list into screen items, `.filter()`, `.find()`.
- **Spread** `{ ...old, bedrooms: 3 }` copies an object and changes one field (React state must be replaced, not edited).
- `async function` with `await fetch(url)` calls the API without freezing the page; `try / catch / finally` handles errors and always clears the "Checking..." state.
- Optional chaining `check?.result` reads a value only if `check` exists; `??` gives a default.
- `import` / `export` split the code into files (modules).

React (version 19):

- A **component** is a function that returns **JSX** (HTML-like syntax inside JavaScript): `function Tips({ items }) { return <ul>...</ul> }`. `{ items }` are the **props** (inputs).
- `useState` keeps values that change (what the user typed, the last check). `useEffect` runs code after drawing, such as fetching the area summary when the area changes; it returns a clean-up function, and a `current` flag ignores out-of-date answers.

- `useMemo` remembers a calculation (the filtered area list) until its inputs change. `useRef` holds the map's HTML element for Leaflet.
- Lists need a `key` (`<li key={place.name}>`) so React can update them efficiently.
- **StrictMode** runs extra checks while developing.

Vite: the development server (`npm run dev`, instant reload on save) and the build tool (`npm run build` makes small files in `dist/`). Settings in `vite.config.js`; `import.meta.env.VITE_API_URL` reads the API address.

npm and package.json: npm downloads packages listed in `package.json` (react, react-dom, leaflet; and vite, @vitejs/plugin-react for development) into `node_modules/`, which is not committed.

Leaflet: an open-source map library. `L.map(element)`, `L.tileLayer(OpenStreetMap tiles URL)`, `L.circleMarker([lat, lon])` with `.bindTooltip(...)`, and `map.fitBounds(points)` to zoom to all markers. OpenStreetMap tiles are free with the credit line shown on the map.

SVG: the Compare histogram and the waterfall bars are drawn by us (rectangles and lines in SVG, or sized `<span>` bars), so no chart library is needed.

CSS: CSS variables (`--ink`, `--gold`) hold the colours; grid and flexbox do the layout; a `@media (max-width: 900px)` rule turns the sidebar into a top bar on phones; `prefers-reduced-motion` turns animations off for users who ask.

Streamlit (backup only): the older version in `streamlit_app/`. Streamlit re-runs the whole Python script on every click and keeps values in `st.session_state`.

6.9 Matplotlib

`plt.figure`, `plt.barh` (a horizontal bar chart), `plt.xlabel`, `plt.title`, `plt.tight_layout` and `plt.savefig(..., dpi=150)`. `matplotlib.use("Agg")` is a non-display backend, so charts are saved to files without opening a window.

6.10 YAML, .env and requirements

- **YAML** (`config.yaml`): a human-readable config format of `key: value` pairs, with indentation for nesting. It is loaded with `yaml.safe_load`.
- **.env:** environment settings (host, port, paths). It is listed in `.gitignore`, so secrets are never committed.
- **requirements.txt:** the list of packages. `pip install -r requirements.txt` installs them all.

6.11 Virtual environment, pip, git and pytest

- **venv:** an isolated Python environment per project, so package versions do not clash with other projects. `python3 -m venv venv` creates it and `source venv/bin/activate` turns it on (Mac/Linux).
- **pip:** the Python package installer.
- **git:** version control. `git status`, `git add`, `git commit -m "..."`, `git push`, `git log`, and branches: `git checkout -b react-frontend` made a separate line of work so the working Streamlit version stayed safe on `main`. `.gitignore` lists files git ignores (venv, `__pycache__`, .env, data/raw). The repo has about 20 commits that tell the project story.
- **pytest:** finds files named `test_*.py` and functions named `test_*`, and runs the `assert` statements. `pytest.raises(Error)` checks that an error happens.

Part 7: How to Run and Demo

7.1 First-time setup

```
cd ~/Real-Estate-AVM
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cd frontend && npm install && cd ..     (the website's packages, once)
```

7.2 Run everything and check it works

```
bash run_checks.sh
```

This runs the model comparison, error measurement, SHAP and the tests, plus a quick API check. The output is also saved to `reports/run_log.txt`.

7.3 Start the app

One command (recommended):

```
bash start_app.sh
```

It starts the backend on port 8000 (with auto-reload), waits until `/health` answers, installs the website's packages the first time, then starts the website on http://localhost:5173. Ctrl+C stops both.

Or two Terminal windows:

```
Window 1:  source venv/bin/activate && uvicorn backend.main:app --reload
Window 2:  cd frontend && npm run dev
```

The API docs are at http://127.0.0.1:8000/docs. The old Streamlit version runs with `bash start_streamlit.sh` (backend must be running).

7.4 Deployment (the link for mentors)

- **Backend on Render:** `render.yaml` describes the service: Python 3.14, install `backend/requirements.txt`, start `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`. A push to GitHub redeploys it.
- **Website on Vercel:** root folder `frontend/`, build `npm run build`, output `dist/` (`vercel.json`). Set `VITE_API_URL` to the Render address.
- **CORS:** set `ALLOWED_ORIGINS` on Render to the Vercel address (or leave it open, since the API only reads public data).
- **Free-tier note:** Render's free server sleeps after a while; the first request can take up to a minute. The website says "Starting up. The first check can take up to 1 minute."

7.5 Demo script (about 5 minutes, practise this)

1. **Open with the problem** (20 sec): "You found a flat online. Is the price fair?"
2. **Check price:** click **Try an example**, then **Fill from ad**. The form fills itself (Whitefield, 1,200 sqft, 2 BHK, 2 bathrooms, ₹1.75 Cr). Point out "Usual price here: ₹14,000 per sqft".
3. Click **Check price**: **"Price is too high"**, "Seller asks 12% more than our price of ₹1.57 Cr". Show the Cheap / Fair / Costly bar with the seller's pin, and the fair price ₹1.41–1.72 Cr.
4. **Tips:** "Ask for a lower price: offer about ₹1.57 Cr, do not pay more than ₹1.94 Cr."

5. **Why this price?** Open it: "Starting from the average flat, size lowers it, bathrooms lower it... these add up exactly to the model's price. That is SHAP. Then prices here went up 2.5 times since the data was collected."
6. **Building age:** type 20, check again: "20-year-old building: about ₹1.48 Cr" and the "Older building" tip.
7. **Typo check:** change the price to 3 Lakhs: "Check the seller's price. Did you mean ₹3 Cr?" Click Yes.
8. **Compare:** the histogram with our price and the seller's price, "costlier than 71% of flats".
9. **Places nearby:** the 5 nearest schools, hospitals and transit with the map; type a society name to measure from it.
10. **Other areas, Before you buy, Loan EMI:** one line each. On Loan EMI, add a salary and show the best plan.
11. **Close with the research result:** "Without location, models cannot beat a simple rule; with it, error drops 18%. Location is about 30% of price."

7.6 If something goes wrong in the demo

- **"Cannot connect":** the backend is not running (start it first), or the online server is waking up (wait a minute).
- **Port already in use:** an old server is still running. Close it, or use another port and set `VITE_API_URL` to match.
- **ModuleNotFoundError:** the venv is not active, or you are not in the project folder.
- **The website looks old after a change:** hard refresh (Cmd+Shift+R).

Part 8: Limitations and Future Work

Examiners respect honesty. Say what the limitation is, why it happens, and how you would fix it.

8.1 Limitations

1. **Listing prices, not sale prices.** The Kaggle data holds asking prices, which may be higher than the final sale prices. The model learns what sellers ask.
2. **Location is at area level, not address level.** Every listing in an area shares the same lat/lon and distances. Two flats in the same area differ only in size, bhk and bath for the model.
3. **No time dimension in the data.** There are no sale dates. We bring prices to 2026 with one factor per area (from published asking rates), which cannot capture differences within an area or between new and old flats.

3b. **Asking-price level.** Both our data and the 2026 rates are asking prices, which usually sit above registered sale prices (often by about a quarter).

4. **Missing amenity data.** 167 of 901 areas have all three distances missing, which probably means the free Overpass server timed out. They were filled with the median, which pretends those areas are typical.
5. **Location names not merged.** 21 near-duplicate names (for example "Banaswadi" and "Banaswadi,") are treated as separate areas, which splits their data.
6. **Geocoding errors.** Nominatim may place an ambiguous name in the wrong spot. 348 of about 1,250 area names could not be found at all, and their 1,245 rows were lost.
7. **Missing features:** floor, condition, parking, facing, carpet vs super built-up, gated society, view. Building age is handled only by a separate rule of thumb (60-year life, ₹2,500/sqft), not learned by the model. `area_type` and `availability` are in the data but not used in the model.

7b. **Geocoding collisions.** A few areas were placed at almost the same point (Kadugodi shows "under 50 m" from Whitefield's centre), so their nearby places are the same.

7c. **No photos, owner or address.** The dataset is anonymous, so we cannot verify listings. That is why the app checks prices instead of listing flats, and gives a "Before you buy" checklist.

8. **Average error is about 24%.** Good enough for a first check, but not a bank-grade valuation.
9. **Trees cannot extrapolate.** For sizes outside 500-5,400 sqft the estimate flattens, so the API warns.
10. **Distance to the *nearest* amenity is crude.** It ignores quality (a good school versus any school) and count (one bus stop versus a metro hub).
11. **Random split, not a spatial or time split.** The same area appears in both train and test, which is realistic for an AVM but optimistic for brand-new areas.

8.2 Future work

- Apply the reviewed name mapping and re-train.
- Re-query the 167 areas with missing amenities using retries (or a paid API), and count amenities within 1 km instead of only the nearest.
- Add metro stations, IT parks and malls, and school quality ratings.
- Use `area_type`, `availability` and `balcony` as features (one-hot encoding).
- Predict `log(price)` with XGBoost, or predict price per sqft, to reduce % error on cheap homes.
- **Prediction intervals** with quantile regression instead of a fixed ± %.
- **Spatial cross-validation** (group k-fold by area) to test truly unseen areas.
- Get dated transaction data (registration records) for time trends.

- Use registered sale prices (Kaveri / sub-registrar records) or a price index to replace the asking-rate factors, and refresh the factors automatically.
- Learn building age, floor and condition from a dataset that has them, instead of a rule of thumb.
- Docker, logging and monitoring for the backend; keep the free server awake.
- Website tests (for example with Playwright) and CI on GitHub Actions to run all tests on every push.
- Remember the last check in the browser, and a shareable link for a check.

Part 9: Viva Question Bank

Practise these out loud. Short, confident answers are best: answer first, then one reason or example. If you do not know, say how you would find out.

9.1 Project and motivation

What is your project in one sentence?

An automated valuation model that predicts Bangalore property prices from size, rooms and location, explains each estimate with SHAP, and tests whether location and amenities really improve prediction.

What is an AVM, and who uses one?

An Automated Valuation Model, a statistical or ML model that estimates property value. Banks use them for loan checks, portals use them for price estimates (like Zillow's Zestimate), and buyers use them to check if an asking price is fair.

What was your research question?

Does adding location and amenity data meaningfully improve price prediction over property attributes alone, and by how much?

What is the answer?

Yes. Without location, both Random Forest and XGBoost are worse than a simple "area rate × size" baseline (RMSE about 59 vs 54.8). Adding location cuts XGBoost's RMSE by 18% (58.95 to 48.28 Lakhs) and raises R^2 from 0.60 to 0.73. SHAP shows location makes up about 30% of the price effect, but it is mostly *which area* (its price level and position), not the distance to the nearest school, hospital or transit stop.

Why Bangalore?

A large, freely licensed dataset exists (Kaggle, 13,320 listings), Bangalore's prices vary a lot between areas, which makes the location question interesting, and OpenStreetMap covers the city well.

Why did you drop the market-surge analysis?

It needs dated, multi-year transaction data. We checked Bangalore, Mumbai and Delhi and none publish it in bulk for free, and our dataset has no sale dates. We documented the scope change and focused on the core question.

What is new or useful in your project?

It does not just predict: it measures the value of location with a controlled experiment, explains each prediction (SHAP), gives a measured error range rather than a single number, and checks whether an asking price is fair.

9.2 Data

Describe the dataset.

Kaggle Bengaluru House Price Data, 13,320 rows and 9 columns: area_type, availability, location, size, society, total_sqft, bath, balcony, price (in Lakhs).

What cleaning did you do?

Extracted bhk from text like "2 BHK". Converted sqft ranges to their average and dropped other units. Dropped rows missing key fields. Removed listings with under 200 sqft per bedroom. Stripped location names. Geocoded the areas and dropped the ones not found. Merged in the amenity distances. Filled missing bath with 0 and distances with the median. Trimmed the 1% tails of price per sqft and size. 13,320 rows became 11,672, then 11,272 for modelling.

Why the 200 sqft-per-bedroom rule?

A bedroom needs at least about 100 sqft and a home has a kitchen, a bathroom and walls too. Under 200 sqft per bedroom is almost certainly a data entry error. The API enforces the same rule.

How did you handle "1200-1400" in total_sqft?

Took the average, 1300. Values in other units (Sq. Meter, Acres, Perch) could not be safely converted, so they became missing and were dropped (46 rows).

Why trim outliers? Why 1% and 99%?

Extreme values were mostly errors, such as a "1,306,800 sqft" flat (land area in the wrong unit). RMSE squares errors, so a few outliers can dominate training and evaluation. 1%/99% removes clear errors while keeping 96.6% of the data (400 rows removed).

Why the median and not the mean for filling distances?

The median is robust to skewed values and outliers. One far-off area would pull the mean up.

What is geocoding? Which service did you use?

Converting a place name to latitude and longitude. We used Nominatim (OpenStreetMap's free geocoder) through geopy, adding ", Bangalore, Karnataka, India" to each name to avoid matching places elsewhere.

Why geocode unique locations and not every row?

About 1,250 unique areas versus about 12,900 rows. At 1 request per second that is about 20 minutes instead of about 3.5 hours, with the same result.

Why time.sleep(1) and sleep(3)?

Nominatim's usage policy is at most 1 request per second, and the Overpass server is a free shared service. Rate limiting is polite and avoids getting blocked.

How did you get amenity distances?

For each area we queried the OpenStreetMap Overpass API for schools, hospitals or clinics, and bus stops or railway stations within 3 km, computed the haversine distance to each, and kept the nearest per category.

What is the haversine formula and why not simple Pythagoras?

It gives the great-circle distance on a sphere: $a = \sin^2(\Delta lat/2) + \cos(lat1)\cos(lat2)\sin^2(\Delta lon/2)$, $d = 2R \cdot \text{asin}(\sqrt{a})$, with R = 6,371 km. Pythagoras on degrees is wrong, because a degree of longitude gets shorter away from the equator and the Earth is curved.

Why are some distances missing?

Either nothing was tagged within 3 km, or the request failed or timed out. 167 areas are missing all three, which suggests failures. We filled them with the median and list this as a limitation.

What did clean_location_names.py find?

Basic cleanup (spaces, commas, capital letters) merged 33 duplicate names. Fuzzy matching (difflib, similarity ≥ 0.85) flagged 163 likely typos, such as Nagondanahalli versus Margondanahalli. We saved them for review instead of auto-merging, because some matches are different places.

How many rows lost at geocoding and why?

1,245 rows (12,917 to 11,672), because 348 area names could not be found by Nominatim.

What does a price of 70 mean?

70 Lakhs, which is Rs 70,00,000, or 7 million rupees. 100 Lakhs = 1 Crore.

9.3 Features and leakage

What features does the final model use?

Nine: bhk, bath, total_sqft_clean (property), plus lat, lon, dist_to_school_km, dist_to_hospital_km, dist_to_transit_km and location_price (location).

What is location_price?

The median price per sqft of the property's area, computed from training rows only. It is a form of target encoding that turns a text category (area name) into a strong number.

Why not one-hot encode the 881 locations?

That would make 881 sparse columns, many areas have only a few rows, and it does not generalise. One target-encoded column holds the useful information compactly. Trees work well with it.

What is data leakage and how did you avoid it?

Leakage is when information the model would not have in real use gets into training. location_price is computed only from the training split. Test-only areas get the overall training median, and a pytest test checks this. Scalers in the linear models sit inside a pipeline, so they are fitted on training data only.

Is there any remaining leakage risk?

The median filling of missing distances uses all rows, but distances are not the target, so the effect is negligible. Also, the same area appears in both train and test, which is realistic for an AVM (we know an area's history) but means scores for brand-new areas could be worse.

Why not use area_type, availability, balcony or society?

The final model focuses on the research question (property vs location). balcony was tried in the earlier scripts. area_type and availability are shown in the app and listed as future features. society is missing for about 40% of rows.

9.4 Models and theory

Why start with a baseline?

To prove the ML model adds value. Our baseline is area median price per sqft × size, which is roughly how a broker estimates. It already reaches R^2 0.65, so beating it is meaningful.

Why did linear regression perform badly?

Price is not a straight-line function of lat/lon or distance. The area effect multiplies with size (rate × sqft), and there are interactions. Even with a log target and regularisation, linear R^2 was only about 0.19 to 0.44, worse than the baseline.

What are Ridge and Lasso?

Linear regression with a penalty on coefficient size. Ridge (L2) shrinks the coefficients, which helps with correlated features. Lasso (L1) can set some to exactly zero, which acts as feature selection.

Why scale features for linear models but not trees?

Regularisation penalises coefficients equally, so the features must be on the same scale. Trees only compare a feature to a threshold, so the scale does not matter.

Why log-transform the target for linear models?

Prices are right-skewed. The log makes the distribution more symmetric and turns multiplicative effects into additive ones, which suits linear models. We reversed it with expm1.

Explain a decision tree.

It repeatedly splits the data with yes/no questions on features, choosing each split to reduce the squared error the most. Each leaf predicts the average target of its rows.

Explain Random Forest.

Many decision trees, each trained on a bootstrap sample of rows, with a random subset of features considered at each split. Predictions are averaged. This reduces the overfitting of single trees (lower variance).

Explain gradient boosting / XGBoost.

Trees are built one after another. Each new tree is fitted to the residual errors of the current ensemble (the gradient of the loss), and its contribution is scaled by the learning rate. XGBoost adds regularisation, handling of missing values, second-order gradients and fast parallel computation.

Bagging vs boosting?

Bagging trains independent models in parallel and averages them, which reduces variance. Boosting trains models in sequence, each correcting the last, which reduces bias.

Why did you pick XGBoost as the final model?

Gradient-boosted trees are the state of the art for tabular data. XGBoost handles non-linear effects and interactions, needs no scaling, is fast, and works with SHAP's exact TreeExplainer. In the comparison it slightly beat Random Forest on RMSE and R^2 in every condition (for example 48.28 vs 49.41 with all features), and the tuned version did better still (RMSE 46.41).

What hyperparameters did you tune, and what do they do?

n_estimators (number of trees), max_depth (tree depth and complexity), learning_rate (step size per tree), subsample (share of rows per tree), colsample_bytree (share of features per tree). The final settings are 100 trees, depth 6, learning rate 0.05, subsample 0.8, colsample 1.0.

Why RandomizedSearchCV instead of GridSearchCV?

The full grid has 1,920 combinations (× 3 folds = 5,760 fits). Randomized search tried 25 (75 fits) and usually finds nearly as good a setting in a fraction of the time.

What is cross-validation and why use it during tuning?

The training data is split into k folds, the model is trained on k-1 and validated on the remaining one, rotating, and the scores are averaged. It gives a more reliable estimate than one split, and it keeps the test set untouched until the final check.

Why is the scoring "neg_root_mean_squared_error"?

sklearn always maximises the score, so error metrics are negated: higher (less negative) is better.

What is overfitting and how did you guard against it?

A model that fits noise in the training data and fails on new data. Guards: a held-out test set, CV during tuning, limited tree depth, subsample, a small learning rate, and saving the tuned model only if it beats the default on test data.

Why random_state=42?

It fixes the randomness (split, bootstrap samples, tree randomness) so results are reproducible. 42 is just a convention; any fixed number works.

Can your model predict a 20,000 sqft villa?

Not reliably. Tree models cannot extrapolate beyond the training range (500-5,400 sqft after trimming): the prediction stays flat. The API returns a warning in that case.

9.5 Metrics and results

Define RMSE, MAE, MAPE and R^2.

MAE is the average absolute error, in Lakhs. RMSE is the square root of the average squared error, which punishes large errors more. MAPE is the average absolute % error. R^2 is the share of variance explained (1 is perfect, 0 is the same as predicting the mean).

Why is RMSE always ≥ MAE?

Squaring gives more weight to big errors, and the square root of a mean of squares is at least the mean of absolute values (a property of means).

Which metric did you use to choose models and why?

RMSE, because large valuation errors are costly and RMSE penalises them more. We also report MAE and MAPE because they are easier to explain.

What is your final model's error?

On the 2,255 test listings: MAPE 23.7%, MAE 23.96 Lakhs, RMSE 46.41 Lakhs, R^2 0.752. 33.7% of estimates are within 10% of the real price and 56.7% within 20%.

Is 24% error good?

For listing-level data with only area-level location and no floor, age or condition, it is reasonable, and better than the baseline. Professional AVMs with address-level data and sale records reach 5-10%. We show it honestly as a range.

Why does the app show a range and not one number?

A single number implies false precision. The range is the estimate ± the measured average % error, so users see the uncertainty. It scales with price.

Why is MAPE problematic sometimes?

It explodes when actual values are near zero, and it penalises over-prediction and under-prediction asymmetrically. Our prices are never near zero (minimum 13 Lakhs after trimming), so it is fine here.

9.6 SHAP and explainability

What is SHAP?

SHapley Additive exPlanations: a method from game theory that splits a prediction into the contributions of each feature. The base value plus the sum of the SHAP values equals the prediction.

What are Shapley values?

From cooperative game theory (Lloyd Shapley): each player's fair share of a payout, averaged over every possible order in which players could join. Here the players are the features and the payout is the prediction.

Why TreeExplainer?

It computes exact SHAP values for tree models quickly. The general method is exponential in the number of features.

What is the base value in your app?

The model's average prediction over the training data. The "Why this price?" card starts from it and each factor adds or subtracts.

What does your global SHAP show?

Size has the largest effect (39.55 Lakhs on average), then area price level (18.28), bathrooms (11.01), bedrooms (3.71). Lat/lon and the three distances are small (1.64 Lakhs or less each). Property is 69.5% and location 30.5%.

Why are bathrooms more important than bedrooms?

bath and bhk are correlated, and bath also signals luxury and larger homes (villas with 4-5 baths), so the model gives bath more of their shared credit. SHAP splits credit between correlated features, so it does not mean bedrooms do not matter.

Does SHAP prove that amenities do not cause higher prices?

No. SHAP explains the model, not cause and effect. Our distances are area-level and vary little, so their effect may be hidden inside location_price and lat/lon.

Global vs local explanation?

Global is the average |SHAP| per feature across all rows (which features matter overall). Local is the SHAP values for one property (why *this* price), which is what the app shows.

9.7 Backend (FastAPI)

Why separate the backend from the frontend?

The model loads once and serves many requests. Any client (web, mobile, other services) can use the API. Each side can be developed, tested and scaled separately. This is separation of concerns.

Why FastAPI?

Automatic request validation with Pydantic, automatic interactive docs at /docs, high performance (ASGI, async), and type-hint based code that is short and clear.

What endpoints do you have?

12: GET /health, /locations, /model-info, /areas, /areas/{area}/summary, /places, /comparison, /nearby; POST /predict, /parse-listing, /locate, /advice; plus the automatic /docs.

Why did you move the area logic from the frontend into the backend?

So the website only displays things. Every number comes from one place (the API), it is tested with pytest, and any other client could reuse it. The logic lives in backend/services/ as plain Python, so it can be tested without a server.

Your SHAP values come from XGBoost, not the shap library. Are they the same?

Yes. booster.predict(..., pred_contribs=True) computes exact TreeSHAP inside XGBoost. A test checks that our base value equals shap.TreeExplainer's. We switched to save memory on the free server.

What is CORS and why do you need it?

The website and the API are on different origins (port 5173 vs 8000, or Vercel vs Render). Browsers block such calls unless the server allows them. FastAPI's CORSMiddleware sends the header that allows it.

Why is /predict a POST and not a GET?

It sends a structured JSON body (property details) to be processed. GET is meant for reading resources, with parameters in the URL.

What happens if someone sends size_sqft = -100?

Pydantic's Field(gt=0) rule fails and FastAPI returns 422 Unprocessable Entity with a message, before our code runs.

What happens for an unknown location?

The API raises HTTPException 404 "Unknown location". It used to return a fake price of 0, which we fixed.

How does the fair-price verdict work?

difference % = (seller's price − today's estimate) / today's estimate × 100. More than +10% is "Price is too high", below −10% "Price is low", otherwise "Price is fair". It compares with **today's** price, because the seller's price is today's. The 10% is set in config.yaml.

Why load the model and data at startup?

Loading is slow. Doing it once means each request only predicts, which takes milliseconds.

Why must feature column order match training?

XGBoost maps input columns to the features it learned. A different order or name would give wrong predictions (or an error), so build_features selects [FEATURES] in the exact order from config.

What is Uvicorn / ASGI?

Uvicorn is a lightweight web server that runs Python ASGI apps. ASGI (Asynchronous Server Gateway Interface) is the standard interface between async Python web apps and servers, the successor to WSGI.

What is a status code 422?

Unprocessable Entity: the request was well-formed JSON but failed validation.

9.8 Frontend (React)

Why did you move from Streamlit to React?

Streamlit is great for quick Python apps, but it re-runs the whole script on every click and limits the design. React gave us real pages, a faster feel, full control of the look (simple language, big clear verdicts, phone layout) and a clean split between website and API, so the website can be hosted on its own.

What is a React component? Give an example from your project.

A function that returns what to show. PriceScale takes our price and the seller's price as props and draws the Cheap / Fair / Costly bar.

What are useState and useEffect?

useState keeps values that change, such as the form fields; changing them redraws the page. useEffect runs code after drawing, for example fetching the area summary when the area changes.

How does the website talk to the backend?

api.js uses fetch with async/await and JSON. The address comes from VITE_API_URL. It waits up to 75 seconds (a sleeping free server) and turns errors into short messages.

How do all pages know about the same flat?

The state lives in App.jsx (lifting state up) and is passed to each page as props. So after a check, Compare, Loan EMI and Before you buy use the same result.

How does page navigation work without reloading?

Hash routing: the page is in the address after #. App listens to the hashchange event and shows the matching page. Back, refresh and links work.

How is the map drawn?

Leaflet with OpenStreetMap tiles: a gold circle for the flat or area centre and coloured circles for schools, hospitals and transit, with tooltips, zoomed to fit.

Why did the map show "API KEY REQUIRED" once?

We first used Carto map tiles, which now need a key. We switched to OpenStreetMap's own tiles, which are free with the credit line.

How did you make the site usable for everyone?

Plain short words, no jargon, the answer first with details one tap away, a Lakhs/Crore switch, a typo check, labels on every input, keyboard support and readable fonts.

What is Vite?

The tool that runs the website while developing (instant reload) and builds small static files for hosting.

9.8b The new features

Your data is from 2017–19. How can you give today's price?

The model learned the pattern from old data. We multiply its answer by how much each area's prices have risen: today's published asking rate (Home Bazaar, Oct 2026) ÷ the area's old median rate, shrunk toward the city factor for small areas, kept between ×1 and ×3. Whitefield is ×2.49; areas without a published rate use the city factor ×1.845.

Why shrink the factors?

Small areas have noisy medians. weight = n / (n + 20) trusts big areas and pulls small ones toward the city figure, so one odd listing cannot double a factor.

Are your "today" prices sale prices?

No, asking-price level, because both our data and the published rates are asking prices. Registered sale prices are usually lower.

Does building age change the model?

No. The data has no age, so the model cannot learn it. We apply the registered valuer rule separately: depreciation = age ÷ 60 × 90%, only on the building's rebuild cost (size × ₹2,500/sqft), because land does not wear out. 20 years means 30%, about ₹9 L for 1,200 sqft. We label it as a rough guide.

Why not let users upload photos?

The model cannot use them (no photos in the training data), so the price would not change, which would be confusing or misleading. It would also need storage and raise privacy questions.

What if someone types 3 instead of 3 Crore?

If the seller's price is under 30% or over 3 times our estimate, we do not judge it. We ask "Did you mean ₹3 Cr?" and re-check with one click. There is also a Lakhs/Crore switch.

How does the Loan EMI page pick the best plan?

It checks the RBI loan-to-value limit (banks lend at most 90% up to ₹30 L, 80% up to ₹75 L, 75% above), then compares 10–30 year loans and picks the shortest with an EMI within 30% of salary, because shorter loans pay much less interest.

How do you read details from a pasted ad?

Regular expressions for BHK, sqft, bathrooms, price in Lakhs or Crore, and building age, and the longest matching area name among the 881. Links cannot be read because property sites block other apps, so users paste the text.

How do you find the nearest named places?

We downloaded all schools, hospitals and transit stops in Bangalore from OpenStreetMap once, then pick the 5 nearest per group within 3 km with haversine. From an exact spot, we geocode the society name with Nominatim first.

Why does the app not list flats for sale?

Our records are anonymous and old: no owner, photos or address, and they cannot be verified. Showing them as for sale would mislead people. So we check prices of flats people find elsewhere, and add a "Before you buy" checklist.

How is it deployed?

Backend on Render (render.yaml), website on Vercel (vercel.json). Pushing to GitHub redeploys both. The website gets the API address from VITE_API_URL.

9.9 Python and code

What is a virtual environment and why use it?

An isolated set of packages for one project, so versions do not clash with other projects or the system Python. It makes the project reproducible with requirements.txt.

What does `if __name__ == "__main__":` do?

The code below it runs only when the file is executed directly, not when it is imported by another file.

What is a decorator? Give one from your code.

A function that wraps another to add behaviour. @app.post("/predict") registers predict() as the handler for POST /predict. @st.cache_data caches a function's output.

What is a lambda?

A small anonymous one-line function, for example lambda f: abs(f.effect) as a sort key.

What is a list comprehension?

A compact way to build a list: [n for n in names if counts[n] > 5].

What is the difference between a list and a tuple?
A list can be changed (mutable); a tuple cannot (immutable). We store (lat, lon) as a tuple because a coordinate pair should not change.

Why replace bare `except:` with a specific exception?
A bare except catches everything, including programming errors and Ctrl+C, which hides bugs. Catching requests.exceptions.RequestException handles only network failures.

What does `.map()` vs `.apply()` do in pandas?
map replaces each value using a dict or Series (an area name to its median rate). apply runs any function on each value (get_bhk on each size string).

What does groupby do?
It splits rows into groups by a key (location), applies a function (median), and combines the results: one value per area.

What is a left join?
It keeps all rows from the left table and adds matching columns from the right. Where there is no match, the values are missing.

What is pickle/joblib and is there a risk?
Python object serialisation, used to save the trained model. Loading a pickle can run code, so only load files you trust.

What is pathlib and why use it?
An object-oriented way to handle file paths that works on any OS. We use Path(__file__) so the app finds its files from any working folder.

What does `python -m backend.models.tune_xgboost` mean?
Run that module as a script, with the project folder on the import path, so package imports like data_processing.prepare_model_data work.

9.10 Testing, tools and process

How did you test the project?
pytest tests for data preparation (row counts, no missing values, the leakage rule, repeatable split) and for the API (valid prediction, range order, SHAP additivity, bigger home costs more, 404/422 handling, verdict thresholds, warnings). run_checks.sh runs every script plus a live API call.

What is the purpose of config.yaml?
A single source of truth for shared settings (seed, split, trim quantiles, features, app settings), so they are consistent across all scripts and easy to change.

Why is .env in .gitignore?
Environment files can hold secrets and machine-specific settings, which should never be committed.

Why commit processed data but not raw data?
The raw data is downloadable from Kaggle. The processed data took hours of geocoding and API calls, so committing it lets anyone run the models immediately.

How did you collaborate?
GitHub with clear ownership (Sherlyn: data and frontend, Manasa: models and backend), small focused commits, and shared modules (prepare_model_data.py, config.yaml) as the agreed interface between the parts.

9.11 Tricky and "what if" questions

If you had one more month, what would you improve first?

Data quality: merge the duplicate area names, re-fetch the missing amenity data, and add metro distance and amenity counts. Then add area_type and availability as features, and use spatial cross-validation.

Could this be used by a bank?

As a first screening tool, yes. For loans, banks need sale-price data, address-level location, property condition and regulatory validation. A 24% average error is too high for a formal valuation.

Is the model biased in any way?

It can reproduce listing-price patterns, including overpricing in popular areas, and it is less reliable for rare property types (very large homes, 6+ BHK, areas with few listings) because there is less training data.

What if a new area is added that is not in the data?

The API returns 404, because there is no lookup row. To support it we would geocode it, fetch amenities, and use the overall median rate (less accurate) until listings exist.

Why not deep learning?

For about 11,000 rows of tabular data, gradient-boosted trees usually match or beat neural networks, train in seconds, need little tuning, and are easier to explain with SHAP.

How do you know the tuned model is better than the default?

Both were scored on the same untouched test set, and the script saves the tuned model only if its RMSE is lower.

What does $R^2 = 0.65$ for the baseline tell you?

Area rate × size alone explains about 65% of price variation, so location and size together are already very informative, which supports our research question.

Why does Random Forest do worse than the baseline without location?

Without any location information, the model cannot tell a 1,200 sqft flat in Koramangala from one on the city edge, while the baseline already uses the area rate. This is direct evidence that location matters.

Why might adding location_price not help Random Forest much on top of lat/lon and distances?

With area-level data, every area has a unique combination of lat, lon and distances, so trees can already identify each area and learn its price level. location_price gives that same information in one ready-made number. This also means part of the "amenity" gain is really "area identity".

Part 10: Cheat Sheet and Glossary

10.1 Numbers to remember

What	Number
Raw listings / columns	13,320 / 9
After sqft, bhk and 200 sqft-per-bedroom checks	12,917
Unique area names geocoded / not found	about 1,250 / 348
Rows after geocoding (final dataset)	11,672 rows, 901 areas
After 1%/99% outlier trim	11,272 rows (400 removed)
Train / test split	9,017 / 2,255 (80/20, random_state 42)
Areas the model knows (lookup)	881
Training size range	500 to 5,400 sqft
Areas missing all 3 distances	167 of 901
Amenity search radius	3 km (3,000 m)
Median distances	school 0.67 km, hospital 0.27 km, transit 0.17 km
Overall training median rate	0.054 Lakhs/sqft = Rs 5,400/sqft
Median price (final data)	70 Lakhs
Location-name duplicates merged by cleanup / typo pairs flagged	33 / 163
Baseline (area rate × size)	RMSE 54.82, MAE 27.72, MAPE 25.6%, R^2 0.654
Best linear model	Lasso + encoded: RMSE 69.53, R^2 0.444 (worse than baseline)
Random Forest property only → full	RMSE 59.17 → 49.41, R^2 0.597 → 0.719
XGBoost property only → full	RMSE 58.95 → 48.28, R^2 0.600 → 0.732
Tuned XGBoost settings	100 trees, depth 6, learning rate 0.05, subsample 0.8, colsample 1.0
Tuning search	25 random combinations × 3-fold CV = 75 fits
Final model MAPE / MAE	23.7% / 23.96 Lakhs
SHAP top 3	size 39.55, area price level 18.28, bathrooms 11.01 (Lakhs average effect)
Property vs location share	69.5% / 30.5%
Fair price band	±10% of today's estimate
Typo check	seller's price under 30% or over 3× our estimate
City market factor / shrinkage	×1.845 / weight = n ÷ (n + 20), factors kept ×1.0–×3.0
Example factors	Whitefield ×2.49, Electronic City ×1.60, Jayanagar ×1.69
2026 rates source	Home Bazaar, 22 Oct 2026: 66 localities, 64 matched
Worked example today	Whitefield 1,200 sqft 2 BHK: ₹1.57 Cr (fair ₹1.41–1.72 Cr)
Building age rule	60-year life, 10% salvage, ₹2,500/sqft; 20 years = 30%
RBI loan limits	90% up to ₹30 L, 80% up to ₹75 L, 75% above
EMI comfort / stretch	30% / 40% of take-home salary
Named places	5 per group within 3 km; 8,448 places in the city
API endpoints / React components / tests	12 / 15 / 73

What	Number
Haversine Earth radius	6,371 km
Nominatim / Overpass delay	1 second / 3 seconds

10.2 Commands

Task	Command
Activate venv	`source venv/bin/activate`
Run all checks	`bash run_checks.sh`
Start everything	`bash start_app.sh` (website on http://localhost:5173)
Start API only	`uvicorn backend.main:app --reload`
Start website only	`cd frontend && npm run dev`
Build website	`cd frontend && npm run build`
Old Streamlit app	`bash start_streamlit.sh`
API docs	http://127.0.0.1:8000/docs
Tests	`python -m pytest -q`
Main experiment	`python -m backend.models.train_random_forest_xgboost`

10.3 Glossary

- **AVM:** Automated Valuation Model, software that estimates property value.
- **Baseline:** the simplest reasonable prediction that a model must beat.
- **Bagging:** training models on random samples and averaging them (Random Forest).
- **Boosting:** training models in sequence, each fixing the previous errors (XGBoost).
- **Cross-validation:** rotating train/validation folds to score a model reliably.
- **Data leakage:** test or target information sneaking into training.
- **Extrapolation:** predicting outside the range of the training data (trees cannot do it).
- **Feature / target:** an input column / the value being predicted.
- **Geocoding:** place name to latitude/longitude.
- **Haversine:** the formula for distance on a sphere.
- **Hedonic pricing:** price as the sum of the values of a property's characteristics.
- **Hyperparameter:** a setting chosen before training (max_depth, learning_rate).
- **Imputation:** filling missing values (median, 0).
- **Lakh / Crore:** 1,00,000 rupees / 100 Lakhs.
- **MAE / RMSE / MAPE / R^2:** absolute error / root mean squared error / % error / variance explained.
- **Overfitting / underfitting:** too complex and memorises / too simple and misses the pattern.
- **Overpass API:** the query service for OpenStreetMap data.
- **Pickle / joblib:** saving Python objects (the model) to a file.
- **Pydantic:** data validation with Python classes.
- **Quantile / percentile:** the value below which a given % of the data lies.
- **Regularisation (L1/L2):** a penalty on model complexity (Lasso/Ridge).
- **REST API:** a web interface using URLs and HTTP methods (GET, POST).
- **SHAP:** SHapley Additive exPlanations, which split a prediction into feature contributions.
- **Component / props / state (React):** a piece of the page, its inputs, and the values it remembers.

- **CORS:** browser rule that blocks calls to another origin unless the server allows them.
- **Time adjustment:** bringing old prices to today with a growth factor.
- **Shrinkage:** pulling a noisy small-sample figure toward an overall figure.
- **Depreciation (straight-line):** a building loses an equal share of its value each year of its life.
- **LTV (loan-to-value):** the share of a home's price a bank may lend.
- **SPA / hash routing:** a one-page website that switches content without reloading, using the part of the address after #.
- **Target encoding:** replacing a category with the mean or median target of that category (location_price).
- **Train/test split:** holding back unseen rows to measure real performance.
- **Uvicorn / ASGI:** the server that runs FastAPI / the async server interface standard.
- **Virtual environment:** an isolated Python package set per project.

> Final tip: when you do not know an answer, say what you do know and how you would find out ("I would check by running... / looking at the SHAP values for..."). Examiners value clear reasoning over memorised facts.